

Universidad Libre Seccional Bogotá
Departamento de Ingeniería de sistemas

Documentación y Gestión del Proyecto
Train Optimización Routes and Navigation System (TORNs)

Presentado por:

Johel Arias
Rebeca Pedrozo
Nicolas Daza

Presentado a:

Ing. Marcela Cifuentes

Arquitectura de la Información 2026-I
Bogotá D. C.

Tabla de Contenido

Capitulo 1. Gestión del Proyecto.....	10
1.1. Planteamiento del problema.....	10
1.2. Justificación del planteamiento del proyecto.....	12
1.3. Marco Teórico y conceptual del proyecto.....	13
1.3.1. Vision por computadora y deteccion de objetos.....	13
1.3.2. Modelo YOLO.....	13
1.3.3. Aprendizaje profundo (Deep Learning).....	13
1.4. Identificación de objetivos generales y específicos del proyecto.....	14
1.4.1. Objetivo General.....	14
1.4.2. Objetivos Específicos.....	14
1.5. Delimitación y alcance del proyecto.....	15
1.5.1. Alcance.....	15
1.5.2. Problemática del Proyecto.....	16
1.6. Factibilidad del proyecto.....	18
1.6.1. Factibilidad técnica.....	18
1.6.2. Factibilidad Operativa.....	19
1.6.3. Factibilidad Económica.....	19
1.6.4. Factibilidad legal.....	20
1.7. Identificación de Stakeholders e interesados.....	20
Tabla 1. Indentificacion de Stakeholders e interesados. Fuente Propia.....	20
1.8. Lista de Actividades del proyecto.....	21
Ilustración 1. Lista de actividades del proyecto. Fuente propia.....	21
1.9. Cronograma del Proyecto.....	22
Ilustración 2. Cronograma del proyecto. Fuente propia.....	22
1.10. Estructura de Desglose del Trabajo.....	23
Ilustración 3. Estructura de Desglose del Trabajo. Fuente propia.....	23
1.11. Matriz de presupuestos del proyecto.....	24
Ilustración 4. Matriz de Presupuestos. Fuente propia.....	24
1.12. Matriz RACI.....	25
Ilustración 5. Matriz RACI. Fuente propia.....	25
1.13. Matriz de Riesgos.....	26
Ilustración 6. Matriz de Riesgos. Fuente propia.....	26
1.14. Entregables principales.....	27
Capitulo 2. Requerimientos del proyecto.....	28

2.1. Requerimientos Funcionales.....	28
2.2. Tablas de descripción textual de Requerimientos Funcionales.....	29
Tabla 2. Tabla de descripción textual. RF-1 Captura de video en estaciones.....	29
Tabla 3. Tabla de descripción textual. RF-2 Detección de personas en tiempo real..	30
Tabla 4. Tabla de descripción textual. RF-3 Generación de métricas de ocupación..	30
Tabla 5. Tabla de descripción textual. RF-4 Optimización de asignación de trenes y vagones.....	31
Tabla 6. Tabla de descripción textual. RF-5 Redistribución en horas pico.....	31
Tabla 7. Tabla de descripción textual. RF-6 Reducción de recursos en horas valle..	32
Tabla 8. Tabla de descripción textual. RF-7Interfaz de monitoreo en tiempo real....	32
Tabla 9. Tabla de descripción textual. RF-8 Visualización gráfica de datos.....	33
Tabla 10. Tabla de descripción textual. RF-9Monitoreo simultáneo de múltiples cámaras.....	33
Tabla 11. Tabla de descripción textual. RF-10 Generación de alertas automáticas...	34
Tabla 12. Tabla de descripción textual. RF-11 Control de acceso al sistema.....	34
Tabla 13. Tabla de descripción textual. RF-12 Dashboard de optimización ferroviaria.....	35
Tabla 14. Tabla de descripción textual. RF-13 Monitoreo de estado del sistema..	35
Tabla 15. Tabla de descripción textual. RF-14 Generación de reportes históricos....	36
2.3. Requerimientos No Funcionales.....	36
2.4. Tablas de descripción textual de Requerimientos No Funcionales.....	37
Tabla 16. Tabla de descripción textual. RNF-1 Escalabilidad por microservicios....	37
Tabla 17. Tabla de descripción textual. RNF-2 Rendimiento en tiempo real.....	37
Tabla 18. Tabla de descripción textual. RNF-3 Confiabilidad y disponibilidad....	38
Tabla 19. Tabla de descripción textual. RNF-4 Seguridad de datos.....	38
Tabla 20. Tabla de descripción textual. RNF-5 Compatibilidad.....	39
Tabla 21. Tabla de descripción textual. RNF-6 Usabilidad.....	39
Tabla 22. Tabla de descripción textual. RNF-7 Mantenibilidad.....	40
2.5. Planteamiento casos de uso de requerimientos funcionales.....	41
Ilustración 7. RF-1. Fuente propia.....	42

Ilustración 8. RF-2. Fuente propia.....	43
Ilustración 9. RF-3. Fuente propia.....	44
Ilustración 10. RF-3. Fuente propia.....	45
Ilustración 11. RF-5. Fuente propia.....	46
Ilustración 12. RF-6. Fuente propia.....	47
Ilustración 13. RF-7. Fuente propia.....	48
Ilustración 14. RF-8. Fuente propia.....	49
Ilustración 15. RF-9. Fuente propia.....	50
Ilustración 17. RF-11. Fuente propia.....	52
Ilustración 18. RF-12. Fuente propia.....	53
Ilustración 19. RF-13. Fuente propia.....	54
Ilustración 20. RF-14. Fuente propia.....	55
2.6. Planteamiento casos de uso de no requirimientos funcionales.....	56
Ilustración 21. RNF-1. Fuente propia.....	56
Ilustración 22. RNF-2. Fuente propia.....	57
Ilustración 23. RNF-3. Fuente propia.....	58
Ilustración 24. RNF-4. Fuente propia.....	59
Ilustración 25. RNF-5. Fuente propia.....	60
Ilustración 26. RNF-6. Fuente propia.....	61
Ilustración 27. RNF-7. Fuente propia.....	62
Capítulo 3. Diagramación de software.....	63
3.1. Diagrama de Flujo de Datos.....	63
Ilustración 28. Diagrama de Flujo de Datos. Fuente propia.....	64
3.2. Diagrama de Arquitectura.....	64
Ilustración 29. Diagrama de Arquitectura Sistema TORNS. Fuente propia.....	65
Capítulo 4. Diseño de Interfaz de Usuario.....	66
4.1. Mapa de navegación.....	66
Ilustración 30. Mapa de Navegacion.....	66
4.2. Mockups.....	67
4.3. Prototipo funcional:.....	67
4.3.1 Home.....	68
4.3.2 Monitoreo.....	68
4.3.3 Cámaras.....	68
4.3.4 Cámaras.....	69
4.3.5 Flujos de Interacción.....	70
Capítulo 5. Arquitectura del sistema.....	71

5.1. YOLO Train Routes Optimization.....	72
5.2. TRO Backend.....	73
5.3. TRO Frontend.....	74
Capitulo 6. Desarrollo - Entrenamiento del modelo.....	75
Ilustración 31. Configuración del entrenamiento del modelo.....	76
6.1. Ejecución del entrenamiento.....	77
Ilustración 32. Epocas de entrenamiento del modelo. Fuente: Propia.....	77
6.2. Imágenes de validación y predicción.....	78
Ilustración 33. Imágenes de validación y predicción. Fuente: Propia.....	78
Ilustración 34. Métricas y curvas de entrenamiento del modelo YOLO a lo largo de las 60 épocas.....	79
Capitulo 7. Base de datos.....	80
7.1 Resumen de tablas.....	80
Tabla 23. Resumen de tablas de la base de datos. Fuente propia.....	80
7.2 Esquema relacional.....	81
7.2.1 train_models.....	81
Tabla 24. Resumen de tabla train_models. Fuente propia.....	81
7.2.1.1 Datos semilla del sistema actual:.....	82
Tabla 25. Datos semilla. Fuente propia.....	82
7.2.2 stations.....	82
Tabla 26. Resumen de tabla stations. Fuente propia.....	82
7.2.2.1 Datos semilla del sistema actual.....	83
Tabla 27. Datos semilla del sistema actual. Fuente propia.....	83
7.2.3 station_train_support.....	83
Tabla 28. Resumen de tabla station_train_support. Fuente propia.....	84
7.2.4 cameras.....	84
Tabla 29. Resumen de tabla cameras. Fuente propia.....	84
7.2.5 detection_events.....	84
Tabla 30. Resumen de tabla detection_events. Fuente propia.....	85
7.2.6 station_states.....	85
Tabla 31. Resumen de tabla station_states. Fuente propia.....	85
7.2.6 dispatch_plans.....	86
Tabla 32. Resumen de tabla dispatch_plans. Fuente propia.....	86
7.3 Diagrama entidad relación.....	87
Ilustración 35. Diagrama entidad relación. Fuente propia.....	87
7.4 Script SQL de Creación.....	88

7.4.1 TABLA: train_models.....	88
Tabla 36. TABLA: train_models. Fuente propia.....	88
7.4.2 TABLA: cameras.....	89
Tabla 37. TABLA: train_models. Fuente propia.....	89
7.4.3 TABLA: stations.....	89
Tabla 38. TABLA: stations. Fuente propia.....	89
7.4.4 TABLA: station_train_support.....	90
Tabla 39. TABLA: station_train_support. Fuente propia.....	90
7.4.5 TABLA: detection_events.....	90
Tabla 40. TABLA: detection_events. Fuente propia.....	90
7.4.6 TABLA: dispatch_plans.....	91
Tabla 41. TABLA: dispatch_plans. Fuente propia.....	91
7.4.7 TABLA: station_states.....	92
Tabla 42. TABLA: station_states. Fuente propia.....	92
7.4.8 Indices de rendimiento.....	93
Tabla 43. TABLA: Indices de rendimiento. Fuente propia.....	93
7.5 Datos semilla - Datos con base en la configuración actual del backend.....	93
7.5.1 Modelos de tren.....	93
Tabla 44. TABLA: Modelos de tren. Fuente propia.....	93
7.5.2 Cámaras.....	94
Tabla 45. TABLA:Cámaras. Fuente propia.....	94
7.5.3 Estaciones.....	94
Tabla 46. TABLA:Estaciones. Fuente propia.....	94
7.5.4 Trenes soportados por cada estación.....	94
Tabla 47. TABLA:Trenes soportados por cada estación. Fuente propia.....	95
7.5.5 Inicializar estados vacíos para cada estación.....	95
Tabla 48. TABLA:Inicializar estados vacíos para cada estación.....	95
7.6 Relaciones entre tablas.....	95
Tabla 49. Resumen de relaciones entre tablas. Fuente propia.....	96
Capítulo 8. Desarrollo - Backend/API.....	97
Capítulo 9. Desarrollo - Frontend / Interfaz visual.....	98
Capítulo 10. Fase de pruebas.....	99
Capítulo 11. Referencias.....	100

Tabla de tablas

Tabla 1. Indentificación de Stakeholders e interesados. Fuente Propia.....	20
Tabla 2. Tabla de descripción textual. RF-1 Captura de video en estaciones.....	29
Tabla 3. Tabla de descripción textual. RF-2 Detección de personas en tiempo real..	30
Tabla 4. Tabla de descripción textual. RF-3 Generación de métricas de ocupación..	30
Tabla 5. Tabla de descripción textual. RF-4 Optimización de asignación de trenes y vagones.....	31
Tabla 6. Tabla de descripción textual. RF-5 Redistribución en horas pico.....	31
Tabla 7. Tabla de descripción textual. RF-6 Reducción de recursos en horas valle..	32
Tabla 8. Tabla de descripción textual. RF-7Interfaz de monitoreo en tiempo real....	32
Tabla 9. Tabla de descripción textual. RF-8 Visualización gráfica de datos.....	33
Tabla 10. Tabla de descripción textual. RF-9Monitoreo simultáneo de múltiples cámaras.....	33
Tabla 11. Tabla de descripción textual. RF-10 Generación de alertas automáticas...	34
Tabla 12. Tabla de descripción textual. RF-11 Control de acceso al sistema.....	34
Tabla 13. Tabla de descripción textual. RF-12 Dashboard de optimización ferroviaria.....	35
Tabla 14. Tabla de descripción textual. RF-13 Monitoreo de estado del sistema..	35
Tabla 15. Tabla de descripción textual. RF-14 Generación de reportes históricos....	36
Tabla 16. Tabla de descripción textual. RNF-1 Escalabilidad por microservicios....	37
Tabla 17. Tabla de descripción textual. RNF-2 Rendimiento en tiempo real.....	37
Tabla 18. Tabla de descripción textual. RNF-3 Confiabilidad y disponibilidad....	38
Tabla 19. Tabla de descripción textual. RNF-4 Seguridad de datos.....	38
Tabla 20. Tabla de descripción textual. RNF-5 Compatibilidad.....	39
Tabla 21. Tabla de descripción textual. RNF-6 Usabilidad.....	39
Tabla 22. Tabla de descripción textual. RNF-7 Mantenibilidad.....	40
Tabla 23. Resumen de tablas de la base de datos. Fuente propia.....	80
Tabla 24. Resumen de tabla train_models. Fuente propia.....	81
Tabla 25. Datos semilla. Fuente propia.....	82

Tabla 26. Resumen de tabla stations. Fuente propia.....	82
Tabla 27. Datos semilla del sistema actual. Fuente propia.....	83
Tabla 28. Resumen de tabla station_train_support. Fuente propia.....	84
Tabla 29. Resumen de tabla cameras. Fuente propia.....	84
Tabla 30. Resumen de tabla detection_events. Fuente propia.....	85
Tabla 31. Resumen de tabla station_states. Fuente propia.....	85
Tabla 32. Resumen de tabla dispatch_plans. Fuente propia.....	86
Tabla 36. TABLA: train_models. Fuente propia.....	88
Tabla 37. TABLA: train_models. Fuente propia.....	89
Tabla 39. TABLA: station_train_support. Fuente propia.....	90
Tabla 40. TABLA: detection_events. Fuente propia.....	90
Tabla 41. TABLA: dispatch_plans. Fuente propia.....	91
Tabla 42. TABLA: station_states. Fuente propia.....	92
Tabla 43. TABLA: Indices de rendimiento. Fuente propia.....	93
Tabla 44. TABLA: Modelos de tren. Fuente propia.....	93
Tabla 45. TABLA:Cámaras. Fuente propia.....	94
Tabla 46. TABLA:Estaciones. Fuente propia.....	94
Tabla 47. TABLA:Trenes soportados por cada estación. Fuente propia.....	95
Tabla 48. TABLA:Inicializar estados vacíos para cada estación.....	95
Tabla 49. Resumen de relaciones entre tablas. Fuente propia.....	96

Tabla de Ilustraciones

Ilustración 1. Lista de actividades del proyecto. Fuente propia.....	22
Ilustración 2. Cronograma del proyecto. Fuente propia.....	23
Ilustración 3. Estructura de Desglose del Trabajo. Fuente propia.....	24
Ilustración 4. Matriz de Presupuestos. Fuente propia.....	25
Ilustración 5. Matriz RACI. Fuente propia.....	26
Ilustración 6. Matriz de Riesgos. Fuente propia.....	27
Ilustración 7. RF-1. Fuente propia.....	43
Ilustración 8. RF-2. Fuente propia.....	44
Ilustración 9. RF-3. Fuente propia.....	45
Ilustración 10. RF-3. Fuente propia.....	46
Ilustración 11. RF-5. Fuente propia.....	47
Ilustración 12. RF-6. Fuente propia.....	48
Ilustración 13. RF-7. Fuente propia.....	49
Ilustración 14. RF-8. Fuente propia.....	50
Ilustración 15. RF-9. Fuente propia.....	51
Ilustración 17. RF-11. Fuente propia.....	53
Ilustración 18. RF-12. Fuente propia.....	54
Ilustración 19. RF-13. Fuente propia.....	55
Ilustración 20. RF-14. Fuente propia.....	56
Ilustración 21. RNF-1. Fuente propia.....	57
Ilustración 22. RNF-2. Fuente propia.....	58
Ilustración 23. RNF-3. Fuente propia.....	59
Ilustración 24. RNF-4. Fuente propia.....	60
Ilustración 25. RNF-5. Fuente propia.....	61
Ilustración 26. RNF-6. Fuente propia.....	62
Ilustración 27. RNF-7. Fuente propia.....	63
Ilustración 28. Diagrama de Flujo de Datos. Fuente propia.....	65
Ilustración 29. Diagrama de Arquitectura Sistema TORNIS. Fuente propia.....	66
Ilustración 30. Mapa de Navegación.....	67
Ilustración 31. Configuración del entrenamiento del modelo.....	77
Ilustración 32. Epocas de entrenamiento del modelo. Fuente: Propia.....	78
Ilustración 33. Imágenes de validación y predicción. Fuente: Propia.....	79
Ilustración 34. Métricas y curvas de entrenamiento del modelo YOLO a lo largo de	

las 60 épocas.....	80
Ilustración 35. Diagrama entidad relación. Fuente propia.....	89

Capítulo 1. Gestión del Proyecto

1.1. Planteamiento del problema.

En una ciudad tan dinámica y extensa como Bogotá, el sistema de transporte masivo constituye uno de los pilares fundamentales para la movilidad de sus habitantes y el desarrollo económico. Sin embargo, a pesar de su importancia estratégica, este sistema enfrenta múltiples desafíos operativos que afectan tanto la eficiencia del servicio como la experiencia diaria de millones de personas e incluso la calidad de vida de todos nosotros.

En el contexto de la futura operación del Metro de Bogotá, la cuál es gestionada por APCA Transmimetro, vemos un escenario particularmente complejo, la necesidad de administrar de manera eficiente un flujo de pasajeros, el cual, puede ser a simple vista una variable dinámica y no tan sencilla de predecir.

A lo largo del día, la demanda de transporte es directamente dependiente a factores como los horarios laborales, estudiantiles, las condiciones del clima y diferentes eventos en la ciudad tales como accidentes, conciertos etc. Todos estos factores generan picos de saturación en determinadas horas y estaciones, tal y como en otros moments puede haber un bajo pico de usuarios.

Uno de los principales retos radica en que actualmente no se cuenta con herramientas tecnologicas suficientemente robustas que permitan anticipar, medir o responder de manera dinamica a todas estas situaciones de picos en la demanda. Es por esto, que muchas de las decisiones operativas se basan en estimaciones historicas o supuestos generales que no necesariamente reflejan el comportamiento real de los usuarios en cada momento.

Dentro del contexto se identifican diversos problemas que impactan directamente la operacion del sistema, tales como:

Fluctuaciones impredecibles de demanda; La afluencia de pasajeros puede variar y en la ausencia de mecanismos automatizados que detecten estos cambios en tiempo real resulta difícil anticipar estas variaciones y preparar la operacion del sistema de manera adecuada.

Ineficiencia en la asignacion de recursos: Al no contar con datos precisos sobre la ocupacion real de las estaciones, la asignación de vagones, trenes y frecuencias se realiza con patrones historicos, lo cual, puede generar un uso ineficiente en horas de baja demanda y saturacion del servicio durante las horas pico.

Tiempos de espera prolongados para los usuarios: La falta de información en tiempo real sobre la cantidad de pasajeros en cada estación limita la posibilidad de ajustar dinamicamente los intervalos entre trenes, por esto, los pasajeros pueden tener tiempos de espera mas largos y mayores niveles de congestion en determinadas estaciones del sistema.

Capacidad desaprovechada: Aunque el sistema cuenta con una gran infraestructura diseñada para movilizar a grandes volumenes de pasajeros, el no tener la informacion operativa basada en datos dificulta aprovechar plenamente dicha capacidad.

Frente a este panorama, surge la necesidad de desarrollar e implementar una solucion tecnologica que permita detectar, medir y analizar en tiempo real la demanda de pasajeros en cada estacion. A parat de esta información, sería posible generar decisiones operativas mas informadas y dinamicas, optimizando la asignación de recursos, reduciendo los tiempos de espera y mejorando significativamente la eficiencia del sistema de transporte y experiencia de los usuarios.

1.2. Justificación del planteamiento del proyecto

El proyecto TORNS es un sistema integral orientado a la optimización operacional de sistemas ferroviarios mediante el uso de técnicas avanzadas de visión por computadora. Su propósito fundamental es detectar, cuantificar y analizar la demanda real de pasajeros en estaciones de metro, con el fin de generar estrategias dinámicas para la asignación eficiente de recursos.

El sistema articula un flujo de datos en tiempo real proveniente de cámaras instaladas en las estaciones, los procesa mediante modelos de detección basados en YOLO y utiliza los resultados para mejorar la planificación operativa a través de un motor de optimización alojado en el backend.

La plataforma busca fortalecer la capacidad de respuesta del sistema de transporte ante fluctuaciones de demanda, minimizando tiempos de espera, mejorando la frecuencia de trenes y ajustando el número de vagones necesarios en función del comportamiento estimado de la afluencia.

Desde una perspectiva técnica, la evolución de los modelos de detección de objetos de la familia You Only Look Once (YOLO) ha demostrado resultados sobresalientes en precisión y velocidad de inferencia, lo que los hace especialmente adecuados para entornos de alta densidad como estaciones de metro. La integración de estos modelos con algoritmos de optimización operativa es una solución viable para este tipo de problemas, y no solo viable sino también escalable y de alto impacto para el transporte público masivo.

1.3. Marco Teórico y conceptual del proyecto.

1.3.1. Vision por computadora y deteccion de objetos

La vision por computadora es una rama de la inteligencia artificial (IA) que permite a los sistemas interpretar y analizar imagenes o videos. Para nuestro contexto, en nuestro proyecto TORNs, esta tecnologia la utilizamos para identificar y contar personas en tiempo real dentro de las estaciones del metro, los modelos de deteccion de objetos, especialmente los basados en redes neuronales convolucionales profundas han alcanzado niveles de precisión muy elevados en tareas de este tipo.

1.3.2. Modelo YOLO

YOLO es una familia de modelos de deteccion de objetos en tiempo real que se caracteriza por procesar imagenes completas en una sola “pasada” o recorrido de la red neuronal, gracias esto es que la red tiene una velocidad de inferencia mayor, en este proyecto utilizamos la version YOLOv11, la cual, contiene mejoras significativas en precision y eficiencia computacional, lo que la hace especialmente adecuada para entornos de alta densidad como las estaciones del metro

1.3.3. Aprendizaje profundo (Deep Learning)

El aprendizaje profundo es un subcampo del aprendizaje automático en donde se utilizan redes neuronales con muchas capas para aprender representaciones jerárquicas de los datos. En nuestro proyecto, esta técnica sustenta tanto el entrenamiento del modelo de detección de personas como los módulos de predicción de demandar a partir de patrones historicos.

1.4. Identificación de objetivos generales y específicos del proyecto

1.4.1. Objetivo General.

Nuestro objetivo general es de implementar una solución automatizada para la detección, cuantificación y análisis de la demanda de pasajeros en estaciones del metro, mediante modelos de aprendizaje profundo basados en visión por computadora, con el fin de optimizar en tiempo real la asignación de recursos operativos del sistema del metro, minimizando tiempos de espera y mejorando la eficiencia en la frecuencia y capacidad de los trenes. Dentro de los objetivos específicos tenemos

1.4.2. Objetivos Específicos.

Investigar y seleccionar los métodos de detección y conteo de personas más adecuados mediante modelos de redes neuronales, evaluando el rendimiento de YOLOv11 en entornos de alta densidad como estaciones del metro

Analizar los requerimientos de integración del sistema con la infraestructura tecnológica existente del metro de Bogotá, identificando restricciones técnicas y operativas

Diseñar la arquitectura técnica del sistema, incluyendo el flujo de datos desde las cámaras hasta el motor de optimización y la interfaz de usuario para el personal operativo

Desarrollar el modelo de red neuronal YOLOv11, la API del backend con algoritmos de optimización y la interfaz visual de monitoreo en tiempo real.

Implementar y validar el sistema en ambientes de prueba y simulación, verificando la precisión del conteo de pasajeros y la efectividad de las decisiones operativas generadas

1.5. Delimitación y alcance del proyecto.

El proyecto TORNS se delimita en el contexto del Metro de Bogotá, específicamente a la primera línea del sistema en operación. El alcance geográfico comprende las estaciones de dicha línea que cuenten con una infraestructura de cámaras compatible con el sistema. En términos temporales, el proyecto tiene una duración de 4 meses, comprendidos entre febrero y mayo del presente año.

El sistema se desarrolla como un prototipo funcional en entorno académico, con proyección de implementación real en colaboración con APCA Transmimetro.

1.5.1. Alcance.

El proyecto comprende las siguientes capacidades funcionales.

Registro y configuración de rutas, estaciones y cámaras del sistema.

Procesamiento de video en tiempo real mediante el modelo YOLOv11 para detección y conteo de pasajeros por estación.

Generación de rutas y frecuencias óptimas a partir de datos históricos, volumen poblacional detectado y niveles de prioridad de cobertura.

Backend robusto con API que expone los resultados de optimización al frontend y permite integración futura con otros sistemas.

Interfaz de usuario que visualiza en tiempo real la ocupación de las estaciones y las recomendaciones operativas generadas.

Vale mencionar que queda fuera del alcance del proyecto los siguientes elementos: la adquisición e instalación física de cámaras en las estaciones, la integración con los sistemas de gestión existentes de APCA Transmimetro.

1.5.2. Problemática del Proyecto

En una ciudad como Bogotá, donde miles de personas dependen del transporte público para movilizarse todos los días, uno de los grandes retos está en lograr que los sistemas masivos funcionen de forma realmente eficiente y respondan a lo que necesitan los usuarios. En el caso del Metro de Bogotá, que próximamente entrará en operación, surge una preocupación importante: la diferencia que existe entre cómo funcionan tradicionalmente los sistemas ferroviarios y lo que exige hoy un sistema de transporte moderno.

La problemática central que busca abordar TORNS tiene que ver precisamente con esa brecha. Muchos sistemas de metro alrededor del mundo siguen operando con horarios y frecuencias previamente establecidos, definidos a partir de estudios históricos de demanda.

Aunque este método ha funcionado durante años, en la práctica no siempre logra adaptarse a lo que realmente ocurre día a día en la ciudad. La movilidad urbana es cambiante, y el comportamiento de los usuarios también.

Cuando el sistema se basa únicamente en estimaciones históricas, empiezan a aparecer varias dificultades. Una de las más evidentes es la congestión durante las horas pico. En ciertos momentos del día, especialmente en las mañanas y al final de la tarde, la cantidad de pasajeros que llega a algunas estaciones puede superar lo que el sistema tenía previsto. Esto genera

incomodidad para los usuarios, aumenta los tiempos de espera y, en algunos casos, incluso puede representar riesgos de seguridad debido a la acumulación de personas en los andenes.

Por otro lado, ocurre el escenario contrario durante las horas valle. En estos periodos de menor demanda, el sistema continúa operando con la misma estructura de recursos, lo que significa que trenes o vagones pueden circular con una ocupación muy baja. Esto implica que se están utilizando más recursos de los realmente necesarios, aumentando los costos operativos sin que exista un beneficio real para el servicio.

A esto se suma otro problema importante: muchas de las decisiones operativas terminan siendo reactivas. Es decir, los operadores del sistema suelen intervenir cuando la congestión o los retrasos ya son evidentes, en lugar de anticiparse a ellos. Esto ocurre, en gran parte, porque no se cuenta con herramientas tecnológicas que permitan identificar con suficiente anticipación cómo está cambiando la demanda de pasajeros en cada estación.

Finalmente, también existe una falta de información en tiempo real. Actualmente no hay un sistema que permita medir de forma continua y automatizada cuántas personas están ingresando o acumulándose en cada estación. Sin estos datos, resulta muy difícil tomar decisiones basadas en información concreta y ajustar la operación del sistema de manera oportuna.

Frente a esta situación, TORNNS surge como una propuesta orientada a cerrar esa brecha. La idea es automatizar el proceso de detección de la demanda de pasajeros y utilizar esos datos para generar recomendaciones operativas que ayuden a mejorar la gestión del sistema. De esta manera, se busca pasar de un modelo donde las decisiones se toman de forma reactiva, a uno en

el que sea posible anticipar los cambios en la demanda y optimizar la operación del metro en función del comportamiento real de los usuarios.

1.6. Factibilidad del proyecto

1.6.1. Factibilidad técnica

Desde el punto de vista técnico, el desarrollo del proyecto es viable porque se apoya en tecnologías que ya han sido ampliamente utilizadas y probadas en diferentes contextos. En el caso del componente de visión por computadora, el modelo YOLOv11 se ha utilizado en múltiples investigaciones y proyectos para la detección de personas en espacios públicos, lo que demuestra que es una herramienta confiable para este tipo de aplicaciones.

Por otra parte, el desarrollo del backend puede realizarse utilizando herramientas bastante comunes dentro del desarrollo de software actual que permiten construir servicios robustos y escalables para procesar la información generada por el sistema.

En cuanto a la parte visual del sistema, la interfaz puede desarrollarse con tecnologías web modernas como React, lo que facilitaría la creación de paneles de monitoreo claros e interactivos para los operadores del sistema.

Además, el equipo de desarrollo cuenta con conocimientos previos en áreas como ingeniería de software, análisis de datos y desarrollo de sistemas, lo cual facilita la implementación del proyecto dentro del tiempo establecido.

1.6.2. *Factibilidad Operativa*

Desde la perspectiva operativa, el sistema fue pensado para integrarse con la infraestructura que ya existe en las estaciones, puede ser integrado con las cámaras de videovigilancia. Esto evita la necesidad de instalar una gran cantidad de nuevos dispositivos, lo que simplifica bastante la implementación.

La plataforma también está diseñada para que el personal operativo pueda utilizarla sin dificultad. La idea es que la interfaz sea clara y sencilla, de modo que los operadores puedan interpretar rápidamente la información sobre la afluencia de pasajeros y tomar decisiones con mayor facilidad.

Adicionalmente, antes de pensar en una implementación completa dentro del sistema, se contempla realizar una fase de pruebas piloto en estaciones específicas. Esto permitiría evaluar el funcionamiento del sistema en un entorno real, identificar posibles ajustes y validar que las recomendaciones generadas realmente aporten valor a la operación.

1.6.3. *Factibilidad Económica.*

En términos económicos, el proyecto tiene un costo estimado de aproximadamente COP \$105.000.000, que incluye gastos asociados a infraestructura tecnológica, desarrollo del software, contratación de personal especializado y algunos costos operativos iniciales.

Aunque puede parecer una inversión considerable, se trata de un monto razonable si se tiene en cuenta el impacto que podría tener en la operación del sistema. Una mejor asignación de trenes, frecuencias y recursos podría ayudar a reducir ineficiencias operativas y optimizar el uso de la infraestructura disponible.

Además, el proyecto cuenta con el respaldo institucional de entidades como la Alcaldía de Bogotá y el Departamento Nacional de Planeación, lo cual fortalece la viabilidad financiera de la propuesta.

1.6.4. Factibilidad legal

El sistema no busca identificar personas ni almacenar información biométrica. En cambio, se enfoca únicamente en contar y analizar la cantidad de pasajeros presentes en las estaciones mediante procesamiento automatizado de video. De esta manera, las imágenes se utilizan con fines operativos y estadísticos, manteniendo el anonimato de los usuarios.

Adicionalmente, el uso de cámaras dentro de sistemas de transporte público es una práctica regulada y permitida en Colombia, siempre que se utilice con fines relacionados con la seguridad, la gestión operativa o la mejora del servicio. En ese sentido, el proyecto se mantiene dentro del marco legal vigente.

1.7. Identificación de Stakeholders e interesados.

Tabla 1. Identificación de Stakeholders e interesados. Fuente Propia

Nombre	Cargo	Departamento / División	Rama ejecutiva (Vicepresidencia)
Departamento Nacional de Planeación	Coordinador de infraestructura	Dirección de ordenamiento territorial	Nacional – Planeación
Secretaría de movilidad	Secretaría distrital de movilidad	Dirección de gestión de transporte masivo	Municipal – Distrital
Apca Transmimetro	Contratista DFBOT (Diseño, Construcción, Finanzas, Operación y Mantenimiento)	Supervisor Técnico y de Control del Proyecto	Sector Privado (Contratista de Obra Pública)

1.8. Lista de Actividades del proyecto.

A continuación se puede encontrar la lista de actividades del proyecto. Esta matriz relaciona todo el detalle a las actividades desarrolladas durante el transcurso del proyecto.

Es posible observar el detalle a través del siguiente enlace.

https://unilibrebog-my.sharepoint.com/:x:/g/personal/johels-ariasb_unilibre_edu_co/IQDMAm-nFkHjRKOZn5q99m18AeDHnByDUHFAXCdrFlsCp2g?e=Ab1BLy

Ilustración 1. Lista de actividades del proyecto. Fuente propia

Nombre del Proyecto

Título del Proyecto		Fecha Inicio		Duración del proyecto en días	
Train Optimization Routes and Navigation System		02/07		113	
Gerente del Proyecto		Fecha Fin			
Johel Santiago Arias Becerra		05/30			

WBS No.	Nombre de la tarea	Estado	Asignado a	Fecha Inicio	Fecha fin	Duración (en días)	Comentarios
1	Fase 1. Documentación y gestión del proyecto			02/07	02/28	21	
1.1.1	Planteamiento del problema	En progreso	Analista de Negocio	02/07	02/09	2	
1.2.1	Justificación del planteamiento del proyecto	En progreso	Analista de Negocio	02/09	02/11	2	
1.3.1	Marco teórico y conceptual del proyecto	No Iniciado	Investigador / Analista	02/11	02/12	1	
1.4.1	Identificación de objetivos generales y específicos del proyecto	Completo	Project Manager	02/12	02/13	1	
1.5.1	Delimitación y alcance del proyecto	Completo	Project Manager	02/13	02/14	1	
1.5.2	Problemática del proyecto	Completo	Analista de Negocio	02/14	02/18	4	
1.5.3	Factibilidad del proyecto	En progreso	Project Manager	02/18	02/21	3	
1.6.1	Identificación de stakeholders e interesados	Completo	Project Manager	02/21	02/23	2	
1.7.1	Elaboración de EDT	En progreso	Project Manager	02/23	02/25	2	
1.7.2	Definir entregables principales	Completo	Project Manager	02/25	02/26	1	
1.7.3	Estimar recursos, costos y presupuesto	No Iniciado	Project Manager	02/26	02/28	2	
2	Fase 2. Requerimientos del proyecto			03/03	03/13	10	
2.1.1	Definición de requerimientos funcionales	No Iniciado	Analista de Requerimientos	03/03	03/04	1	
2.1.2	Creación de tablas de requerimientos funcionales	No Iniciado	Analista de Requerimientos	03/04	03/06	2	
2.1.3	Definición de requerimientos no funcionales	No Iniciado	Analista de Requerimientos	03/06	03/07	1	
2.1.4	Creación de tablas de requerimientos no funcionales	No Iniciado	Analista de Requerimientos	03/07	03/09	2	
2.2.1	Planteamiento de casos de uso de requerimientos funcionales	No Iniciado	Analista de Sistemas	03/09	03/11	2	
2.2.4	Planteamiento de casos de uso de requerimientos no funcionales	No Iniciado	Analista de Sistemas	03/11	03/13	2	
3	Fase 3. Diagramación de software			03/16	03/22	6	
3.1.1	Diagrama de flujo de datos	No Iniciado	Arquitecto de Software	03/16	03/18	2	
3.1.4	Diagrama de arquitectura	No Iniciado	Arquitecto de Software	03/18	03/19	1	
3.1.6	Diagrama entidad-relación	No Iniciado	Arquitecto de Software / Ingeniero de Datos	03/19	03/22	3	
4	Fase 4. Diseño de Interfaz de Usuario			03/25	04/03	9	
4.1.1	Mapa de navegación	No Iniciado	Diseñador UX	03/25	03/26	1	
4.2.1	Mockups	No Iniciado	Diseñador UX	03/26	03/29	3	
4.3.1	Prototipo funcional	No Iniciado	Diseñador UX/UI	03/29	04/03	5	
5	Fase 5. Desarrollo - Entrenamiento del modelo			04/06	04/15	9	
5.1.1	Identificación y búsqueda de datasets	Completo	Ingeniero de Datos	04/06	04/07	1	

1.9. Cronograma del Proyecto.

A continuación se puede encontrar el cronograma del proyecto. Esta matriz relaciona todo el detalle correspondiente a las fechas de realización de cada actividad durante el transcurso del proyecto.

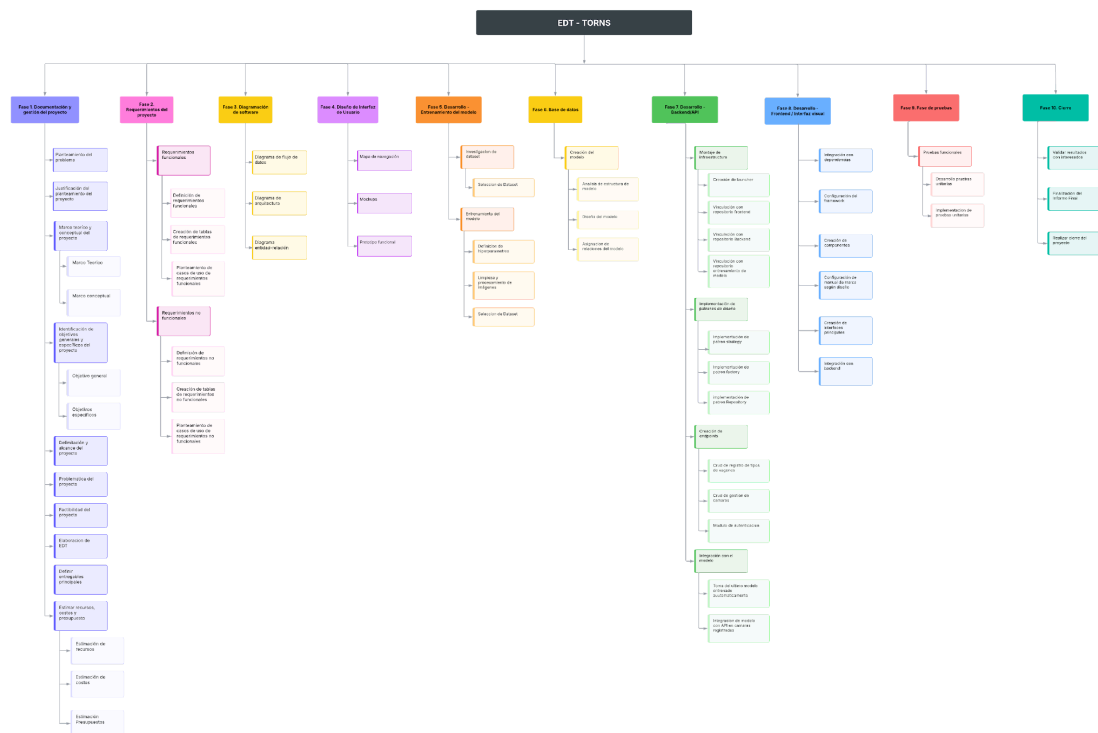
Es posible observar el detalle a través del siguiente enlace.

https://unilibrebog-my.sharepoint.com/:x:/g/personal/johels-ariasb_unilibre_edu_co/IQD736giX_VLOKhVpPZwJhKvAY5xXtUjf73r3Bmv4j_8Dps?e=yurdG5

Ilustración 2. Cronograma del proyecto. Fuente propia

Título del Proyecto		Fecha inicio		Duración del proyecto en días	
Train Optimization Routes and Navigation System		02/07		113	
Gerente del Proyecto		Fecha Fin			
Johel Santiago Arias Becerra		05/30			

WBS No.	Nombre de la tarea	Estado	Asignado a	Fecha Inicio	Fecha fin	Duración (en días)	Comentarios
1	Fase 1. Documentación y gestión del proyecto			02/07	02/28	21	
1.1.1	Planteamiento del problema	En progreso	Analista de Negocio	02/07	02/09	2	
1.2.1	Justificación del planteamiento del proyecto	En progreso	Analista de Negocio	02/09	02/11	2	
1.3.1	Marco teórico y conceptual del proyecto	No Iniciado	Investigador / Analista	02/11	02/12	1	
1.4.1	Identificación de objetivos generales y específicos del proyecto	Completo	Project Manager	02/12	02/13	1	
1.5.1	Delimitación y alcance del proyecto	Completo	Project Manager	02/13	02/14	1	
1.5.2	Problemática del proyecto	Completo	Analista de Negocio	02/14	02/18	4	
1.5.3	Factibilidad del proyecto	En progreso	Project Manager	02/18	02/21	3	
1.6.1	Identificación de stakeholders e interesados	Completo	Project Manager	02/21	02/23	2	
1.7.1	Elaboración de EDT	En progreso	Project Manager	02/23	02/25	2	
1.7.2	Definir entregables principales	Completo	Project Manager	02/25	02/26	1	
1.7.3	Estimar recursos, costos y presupuesto	No Iniciado	Project Manager	02/26	02/28	2	
2	Fase 2. Requerimientos del proyecto			03/03	03/13	10	
2.1.1	Definición de requerimientos funcionales	No Iniciado	Analista de Requerimientos	03/03	03/04	1	
2.1.2	Creación de tablas de requerimientos funcionales	No Iniciado	Analista de Requerimientos	03/04	03/06	2	
2.1.3	Definición de requerimientos no funcionales	No Iniciado	Analista de Requerimientos	03/06	03/07	1	
2.1.4	Creación de tablas de requerimientos no funcionales	No Iniciado	Analista de Requerimientos	03/07	03/09	2	
2.2.1	Planteamiento de casos de uso de requerimientos funcionales	No Iniciado	Analista de Sistemas	03/09	03/11	2	
2.2.2	Planteamiento de casos de uso de requerimientos no funcionales	No Iniciado	Analista de Sistemas	03/11	03/13	2	
3	Fase 3. Diagramación de software			03/16	03/22	6	
3.1.1	Diagrama de flujo de datos	No Iniciado	Arquitecto de Software	03/16	03/18	2	
3.1.4	Diagrama de arquitectura	No Iniciado	Arquitecto de Software	03/18	03/19	1	
3.1.6	Diagrama entidad-relación	No Iniciado	Arquitecto de Software / Ingeniero de Datos	03/19	03/22	3	
4	Fase 4. Diseño de Interfaz de Usuario			03/25	04/03	9	
4.1.1	Mapa de navegación	No Iniciado	Diseñador UX	03/25	03/26	1	
4.2.1	Mockups	No Iniciado	Diseñador UX	03/26	03/29	3	
4.3.1	Prototipo funcional	No Iniciado	Diseñador UX/UI	03/29	04/03	5	
5	Fase 5. Desarrollo - Entrenamiento del modelo			04/06	04/15	9	
5.1.1	Identificación y búsqueda de datasets	Completo	Ingeniero de Datos	04/06	04/07	1	



Presupuesto de Proyecto																			
Proyecto TORRES			Johel Beltrán Roberto Pedraza Humberto Torres			Mano de Obra			Materiales			Costos Fijos			Balance				
TAREA	DESCRIPCION	ESTADO	FECHA INICIO PLANIFICA	FECHA FIN PLANIFICA	FECHA FIN REAL	HORAS	VALOR HORA	TOTAL MANO DE OBRA	UNIDADES	VALOR UNIDAD	TOTAL MATERIALES	VIAJES	EMPEÑO	MISCELÁNEOS	PRESPUESTO	REAL	Diferencia (Presup-Real)		
TORRES, Torre Polivalente Residencial y Multiplaza Sector																			
Fase 1. Documentación y calidad del proyecto																			
Definición del problema	Planteamiento del problema	En progreso	02/07	02/07	02/08	10,0	\$10.000,00	\$100.000,00	1,0	\$200,00	\$200,00	\$500,00	\$20.000,00	\$0,00	\$10.000,00	\$20.000,00	\$0,00	\$0,00	
Justificación del planteamiento del problema	Justificación del planteamiento del problema	En progreso	02/08	02/08	02/09	1,0	\$10.000,00	\$10.000,00	1,0	\$200,00	\$200,00	\$500,00	\$0,00	\$0,00	\$10.000,00	\$0,00	\$10.000,00	\$0,00	
Planteamiento y alcance del problema	Planteamiento y alcance del problema	Hecho	02/09	02/09	02/10	1,0	\$10.000,00	\$10.000,00	1,0	\$200,00	\$200,00	\$500,00	\$0,00	\$0,00	\$10.000,00	\$0,00	\$10.000,00	\$0,00	
Identificación de objetivos generales y específicos del proyecto	Identificación de objetivos generales y específicos del proyecto	Completado	02/10	02/10	02/11	1,0	\$10.000,00	\$10.000,00	1,0	\$200,00	\$200,00	\$500,00	\$0,00	\$0,00	\$10.000,00	\$0,00	\$10.000,00	\$0,00	
Definición y alcance del problema	Definición y alcance del problema	Completado	02/10	02/10	02/11	1,0	\$10.000,00	\$10.000,00	1,0	\$200,00	\$200,00	\$500,00	\$0,00	\$0,00	\$10.000,00	\$0,00	\$10.000,00	\$0,00	
Problematización del problema	Problematización del problema	Completado	02/10	02/10	02/11	1,0	\$10.000,00	\$10.000,00	1,0	\$200,00	\$200,00	\$500,00	\$0,00	\$0,00	\$10.000,00	\$0,00	\$10.000,00	\$0,00	
Factibilidad del problema	Factibilidad del problema	En progreso	02/10	02/10	02/11	1,0	\$10.000,00	\$10.000,00	1,0	\$200,00	\$200,00	\$500,00	\$0,00	\$0,00	\$10.000,00	\$0,00	\$10.000,00	\$0,00	
Identificación de stakeholders e interesados	Identificación de stakeholders e interesados	Completado	02/11	02/11	02/12	1,0	\$10.000,00	\$10.000,00	1,0	\$200,00	\$200,00	\$500,00	\$0,00	\$0,00	\$10.000,00	\$0,00	\$10.000,00	\$0,00	
Elaboración de EOT	Elaboración de EOT	En progreso	02/12	02/12	02/01	2,0	\$10.000,00	\$20.000,00	1,0	\$200,00	\$200,00	\$500,00	\$0,00	\$0,00	\$20.000,00	\$0,00	\$20.000,00	\$0,00	
Definir roles y responsabilidades	Definir roles y responsabilidades	Completado	02/01	02/01	02/02	1,0	\$10.000,00	\$10.000,00	1,0	\$200,00	\$200,00	\$500,00	\$0,00	\$0,00	\$10.000,00	\$0,00	\$10.000,00	\$0,00	
Estimar recursos, costos y presupuesto	Estimar recursos, costos y presupuesto	En progreso	02/02	02/02	02/03	3,0	\$10.000,00	\$30.000,00	1,0	\$200,00	\$200,00	\$500,00	\$0,00	\$0,00	\$30.000,00	\$0,00	\$30.000,00	\$0,00	
Fase 2. Organización del proyecto																			
Definición de responsabilidades funcionales	Definición de responsabilidades funcionales	Hecho	03/03	03/03	03/04	10,0	\$10.000,00	\$100.000,00	5,0	\$2.000,00	\$2.000,00	\$0,00	\$0,00	\$0,00	\$100.000,00	\$0,00	\$100.000,00	\$0,00	
Creación de roles de responsabilidades funcionales	Creación de roles de responsabilidades funcionales	Hecho	03/04	03/04	03/05	1,0	\$10.000,00	\$10.000,00	1,0	\$200,00	\$200,00	\$500,00	\$0,00	\$0,00	\$10.000,00	\$0,00	\$10.000,00	\$0,00	
Definición de responsabilidades no funcionales	Definición de responsabilidades no funcionales	Hecho	03/05	03/05	03/06	1,0	\$10.000,00	\$10.000,00	1,0	\$200,00	\$200,00	\$500,00	\$0,00	\$0,00	\$10.000,00	\$0,00	\$10.000,00	\$0,00	
Creación de roles de responsabilidades no funcionales	Creación de roles de responsabilidades no funcionales	Hecho	03/06	03/06	03/07	1,0	\$10.000,00	\$10.000,00	1,0	\$200,00	\$200,00	\$500,00	\$0,00	\$0,00	\$10.000,00	\$0,00	\$10.000,00	\$0,00	
Planteamiento de acciones de responsabilidades no funcionales	Planteamiento de acciones de responsabilidades no funcionales	Hecho	03/07	03/07	03/08	1,0	\$10.000,00	\$10.000,00	1,0	\$200,00	\$200,00	\$500,00	\$0,00	\$0,00	\$10.000,00	\$0,00	\$10.000,00	\$0,00	
Planteamiento de acciones de responsabilidades no funcionales	Planteamiento de acciones de responsabilidades no funcionales	Hecho	03/08	03/08	03/09	3,0	\$10.000,00	\$30.000,00	1,0	\$200,00	\$200,00	\$500,00	\$0,00	\$0,00	\$30.000,00	\$0,00	\$30.000,00	\$0,00	
Planteamiento de acciones de responsabilidades no funcionales	Planteamiento de acciones de responsabilidades no funcionales	Hecho	03/09	03/09	03/10	3,0	\$10.000,00	\$30.000,00	1,0	\$200,00	\$200,00	\$500,00	\$0,00	\$0,00	\$30.000,00	\$0,00	\$30.000,00	\$0,00	
Fase 3. Diagramas de análisis																			
Diagrama de flujo de datos	Diagrama de flujo de datos	Hecho	03/10	03/10	03/12	8,0	\$75.000,00	\$600.000,00	1,0	\$4.500,00	\$4.500,00	\$0,00	\$0,00	\$0,00	\$600.000,00	\$0,00	\$600.000,00	\$0,00	
Diagrama de flujo de control	Diagrama de flujo de control	Hecho	03/10	03/10	03/11	3,0	\$25.000,00	\$75.000,00	1,0	\$250,00	\$250,00	\$500,00	\$0,00	\$0,00	\$75.000,00	\$0,00	\$75.000,00	\$0,00	
Diagrama de arquitectura	Diagrama de arquitectura	Hecho	03/10	03/10	03/11	2,0	\$25.000,00	\$50.000,00	1,0	\$250,00	\$250,00	\$500,00	\$0,00	\$0,00	\$50.000,00	\$0,00	\$50.000,00	\$0,00	
Diagrama de entidad-relación	Diagrama de entidad-relación	Hecho	03/10	03/10	03/12	3,0	\$25.000,00	\$75.000,00	1,0	\$250,00	\$250,00	\$500,00	\$0,00	\$0,00	\$75.000,00	\$0,00	\$75.000,00	\$0,00	
Fase 4. Diseño de implementación																			
Mapa de navegación	Mapa de navegación	Hecho	03/12	03/12	03/13	10,0	\$75.000,00	\$750.000,00	1,0	\$10.000,00	\$10.000,00	\$0,00	\$0,00	\$0,00	\$750.000,00	\$0,00	\$750.000,00	\$0,00	
Mapas de datos	Mapas de datos	Hecho	03/12	03/12	03/13	3,0	\$25.000,00	\$75.000,00	1,0	\$10.000,00	\$10.000,00	\$0,00	\$0,00	\$0,00	\$75.000,00	\$0,00	\$75.000,00	\$0,00	
Prototipo funcional	Prototipo funcional	Hecho	03/12	03/12	03/13	10,0	\$25.000,00	\$250.000,00	2,0	\$10.000,00	\$20.000,00	\$20.000,00	\$0,00	\$0,00	\$0,00	\$20.000,00	\$0,00	\$20.000,00	\$0,00
Fase 5. Documentación y calidad del modelo																			
Identificación y chequeo de datos	Identificación y chequeo de datos	Completado	04/01	04/01	04/02	10,0	\$100.000,00	\$1.000.000,00	1,0	\$10.000,00	\$10.000,00	\$5.000,00	\$0,00	\$0,00	\$1.000.000,00	\$0,00	\$1.000.000,00	\$0,00	
Definición de interoperabilidad	Definición de interoperabilidad	Completado	04/02	04/02	04/03	1,0	\$20.000,00	\$20.000,00	1,0	\$5.000,00	\$5.000,00	\$5.000,00	\$0,00	\$0,00	\$10.000,00	\$0,00	\$10.000,00	\$0,00	
Límites y normalización de información	Límites y normalización de información	Completado	04/03	04/03	04/05	3,0	\$40.000,00	\$120.000,00	1,0	\$40.000,00	\$40.000,00	\$20.000,00	\$0,00	\$20.000,00	\$0,00	\$20.000,00	\$0,00	\$20.000,00	\$0,00

1.12. Matriz RACI.

A continuación se puede encontrar la matriz RACI del proyecto. Esta matriz relaciona todo el detalle correspondiente a la asignación de responsabilidades de las actividades del proyecto.

Es posible observarlo a detalle a través del siguiente enlace.

https://unilibrebog-my.sharepoint.com/:x:/g/personal/johels-ariasb_unilibre_edu_co/IOA3_ZmES9TvTqNbtx8LuEFBAVG_sfM0jlFHyLrgQUGwuHw?e=eT1o9F

Ilustración 5. Matriz RACI. Fuente propia

Johel Santiago Arias
Rebecca Pedraza
Nicolas Diaz Redondo

		Abordador del proyecto											
		Project Manager											
		Líder Técnico											
		Líder de Creación de Datos											
		Analistas											
		Programador de Vision Artificial											
		Programador de Datos											
		Programador Apilados											
		Programador de Infraestructura											
		Programador Financiera											
		Ejecutor en Dominio Financiero											
		Ejecutora Ops											
ESTADO	RESULTADO/ACTIVIDAD DEL PROYECTO	Liderazgo de proyectos					Equipo Técnico					Operativos	
	Fase 1. Documentación y gestión del proyecto												
En progreso	Ponenteamiento del problema	I	A	R		C						C	
En progreso	Justificación del planteamiento del proyecto	I	A	R		C						C	I
No iniciado	Marco teórico y conceptual del proyecto	I	A	R		C						C	
Completado	Identificación de objetivos generales y específicos del proyecto	I	A	R		C	I	I	I	I	I	C	A
Completado	Delimitación y alcance del proyecto	I	A	R		C	I	I	I	I	I	C	A
Completado	Problemática del proyecto	I	A	R		C						C	
En progreso	Factibilidad del proyecto	I	A	R		C						C	
Completado	Identificación de stakeholders e interesados	I	A	R		C						C	
En progreso	Elaboración de EOI		I	A	C		I	I	I	I	I		C
Completado	Definir entregables principales		A	R	C								C
En progreso	Estimar recursos, costos y presupuesto		A	R	C								C
	Fase 2. Requerimientos del proyecto												
No iniciado	Definición de requerimientos funcionales	I	I	A	R	C	R	R	R	R	R	C	I
No iniciado	Creación de tablas de requerimientos funcionales			A	R		R	R	R	R	R		
No iniciado	Definición de requerimientos no funcionales	I	I	A	R	C	R	R	R	R	R	C	I
No iniciado	Creación de tablas de requerimientos no funcionales			A	R		R	R	R	R	R		
No iniciado	Ponenteamiento de casos de uso de requerimientos funcionales			A	R		R	R	R	R	R		
No iniciado	Ponenteamiento de casos de uso de requerimientos no funcionales			A	R		R	R	R	R	R		
	Fase 3. Diagramación de software												
No iniciado	Diagrama de flujo de datos			A	I		I	I	R	R	I		
No iniciado	Diagrama de arquitectura			A	I		I	I	R	R	I		
No iniciado	Diagrama entidad-relación			A	I		I	I	R	R	I		
	Fase 4. Diseño de Interfaz de Usuario												
No iniciado	Mapa de navegación		I	A	I	I		I	I	I	R	I	I
No iniciado	Mockups		I	A	I	I		I	I	I	R	I	I
No iniciado	Prototipo funcional		I	A	I	I		I	I	I	R	I	I
	Fase 5. Desarrollo - Entrenamiento del modelo												
Completado	Identificación y búsqueda de datasets		I	I	A		R	R	I				I
Completado	Definición de hiperparámetros		I	I	A		R	C	I	I			I
Completado	Limpieza y procesamiento de imágenes		I	I	A		R	C	C				I
Completado	Entrenamiento del modelo		I	I	A		R	R	I	I			I
	Fase 6. Base de datos												
No iniciado	Creación del modelo			A	C		I	C	R	I			I

1.13. Matriz de Riesgos.

A continuación se puede encontrar la matriz de riesgos del proyecto. Esta matriz relaciona todo el detalle correspondiente a los posibles riesgos y medidas de mitigación de las actividades con mayor probabilidad de riesgo en el proyecto.

Es posible observarlo a detalle a través del siguiente enlace.

https://unilibrebog-my.sharepoint.com/:x:/g/personal/johels-ariasb_unilibre_edu_co/IQD_292T0JunRZMFYL-fKBiFAaD_UYg42UUI3rPuDzM4lyM?e=8BdqDi

Ilustración 6. Matriz de Riesgos. Fuente propia

Evaluación de Riesgos del Proyecto									
B. del Proyecto/tema del Proyecto						Sistema del Proyecto			
TCRNS Train Optimization Routes and Navigation System									

1.14. Entregables principales.

El avance del proyecto esta constituido a través de los entregables definidos. Estos entregables están compuestos por los siguientes elementos.

Documento. Analisis, diseño, documentacion y gestion del Proyecto.

Formatos. Lista de Actividades, Cronograma, EDT, Presupuestos, RACI, Riesgos.

Repositorios deCodigo Fuente. Launcher, Entrenamiento del modelo, Frontend,
Backend.

Base de Datos. Base de datos del proyecto

Capítulo 2. Requerimientos del proyecto

2.1. Requerimientos Funcionales

El proyecto tiene una serie de requerimientos funcionales

RF-1 Captura de video en estaciones: El sistema debe ser capaz de recibir transmisión continua de cámaras IP.

RF-2 Detección de personas en tiempo real: El sistema debe identificar y contar pasajeros en cada fotograma mediante modelos YOLO.

RF-3 Generación de métricas de ocupación: El sistema debe calcular volumen de usuarios y niveles de ocupación por estación.

RF-4 Optimización de asignación de trenes y vagones: El sistema debe calcular la distribución óptima de trenes y vagones según la demanda detectada.

RF-5 Redistribución en horas pico: El sistema debe asignar trenes adicionales cuando se detecte alta afluencia de pasajeros.

RF-6 Reducción de recursos en horas valle: El sistema debe disminuir la cantidad de trenes y vagones en periodos de baja demanda.

RF-7 Interfaz de monitoreo en tiempo real: El sistema debe mostrar video en vivo, métricas y alertas en un dashboard para operadores.

RF-8 Visualización gráfica de los datos: El sistema debe presentar métricas mediante gráficos interactivos en el frontend.

RF-9 Monitoreo simultáneo en múltiples cámaras: El sistema debe permitir visualizar varias cámaras en paralelo con baja latencia.

RF-10 Generacion de alertas automáticas: El sistema debe notificar a operadores cuando se detecten aglomeraciones o sobrecargas.

RF-11 Control de accesos al sistema: El sistema debe permitir autenticación y autorización diferenciada por roles.

RF-12 Dashboard de optimización ferroviaria: El sistema debe mostrar resultados de asignación óptima de trenes en la interfaz.

RF-13 Monitoreo y estado del sistema: El sistema debe indicar cámaras activas, objetos detectados y última actualización.

RF-14 Gneracion de reportes históricos: El sistema debe permitir acceso a datos pasados de ocupación y flujo.

2.2. Tablas de descripción textual de Requerimientos Funcionales.

Tabla 2. Tabla de descripción textual. RF-1 Captura de video en estaciones.

REQUERIMIENTOS FUNCIONALES					
RF-1 Captura de video en estaciones	RF-1	Necesidad relacionada	Prioridad 9/10	Captura de video en estaciones	Recepción de transmisión de video.
Descripción	El sistema debe recibir transmisión continua desde cámaras IP instaladas en las estaciones, garantizando compatibilidad con protocolos estándar y reconexión automática ante fallos.				
Justificación	La captura de video es esencial para alimentar el módulo de detección y análisis en tiempo real.				
Origen (Interesado)	Cliente (Operador).				
Criterio de aceptación / Validación	- El sistema recibe video en vivo de cámaras IP conectadas. - Compatible con RTSP/HTTP/ONVIF. - Flujo disponible sin interrupciones >99%.				

Tabla 3. Tabla de descripción textual. RF-2 Detección de personas en tiempo real.

REQUERIMIENTOS FUNCIONALES					
RF-2 Detección de personas en tiempo real	RF-2	Necesidad relacionada	Prioridad 10/10	Detección de personas en tiempo real	Detección y conteo de personas.
Descripción	El sistema debe identificar y contar pasajeros en cada fotograma mediante modelos YOLO, con precisión mínima del 80% en pruebas de validación.				
Justificación	Permite conocer la afluencia de pasajeros y alimentar la optimización ferroviaria.				
Origen (Interesado)	Cliente (Operador).				
Criterio de aceptación / Validación	• Detecta y contabiliza personas con precisión $\geq 80\%$. • Procesamiento en tiempo real ($< 1s$ de latencia). • Conteo actualizado por fotograma.				

Tabla 4. Tabla de descripción textual. RF-3 Generación de métricas de ocupación.

REQUERIMIENTOS FUNCIONALES					
RF-3 Generación de métricas de ocupación	RF-3	Necesidad relacionada	Prioridad 9/10	Generación de métricas de ocupación	Cálculo de métricas de ocupación.
Descripción	El sistema debe calcular volumen de usuarios y niveles de ocupación por estación, actualizando métricas periódicamente.				
Justificación	Las métricas son necesarias para tomar decisiones operativas basadas en datos reales.				
Origen (Interesado)	Cliente (Operador).				
Criterio de aceptación / Validación	• Se generan métricas de ocupación por estación. • Clasificación en niveles (bajo, medio, alto). • Actualización ≤ 1 min.				

Tabla 5. Tabla de descripción textual. RF-4 Optimización de asignación de trenes y vagones.

REQUERIMIENTOS FUNCIONALES					
RF-4 Optimización de asignación de trenes y vagones	RF-4	Necesidad relacionada	Prioridad 10/10	Optimización de asignación de trenes y vagones	Cálculo de asignación óptima.
Descripción	El sistema debe calcular la distribución óptima de trenes y vagones según la demanda detectada, considerando capacidad máxima.				
Justificación	Garantiza eficiencia operativa y evita sobrecargas.				
Origen (Interesado)	Cliente (Operador).				
Criterio de aceptación / Validación	• Algoritmo calcula asignación óptima. • Resultados exportables al dashboard. • Ajustes reflejados en ≤ 2 min.				

Tabla 6. Tabla de descripción textual. RF-5 Redistribución en horas pico.

REQUERIMIENTOS FUNCIONALES					
RF-5 Redistribución en horas pico	RF-5	Necesidad relacionada	Prioridad 9/10	Redistribución en horas pico	Redistribución de trenes.
Descripción	El sistema debe asignar trenes adicionales automáticamente cuando se detecte alta afluencia de pasajeros.				
Justificación	Reduce tiempos de espera y evita saturación.				
Origen (Interesado)	Cliente (Usuario final).				
Criterio de aceptación / Validación	• Se detecta alta afluencia en tiempo real. • Se asignan trenes adicionales. • Ajustes visibles en dashboard.				

Tabla 7. Tabla de descripción textual. RF-6 Reducción de recursos en horas valle.

REQUERIMIENTOS FUNCIONALES					
RF-6 Reducción de recursos en horas valle	RF-9	Necesidad relacionada	Prioridad 8/10	Reducción de recursos en horas valle	Reducción de trenes/vagones.
Descripción	El sistema debe disminuir la cantidad de trenes y vagones en periodos de baja demanda.				
Justificación	Optimiza costos operativos.				
Origen (Interesado)	Cliente (Operador).				
Criterio de aceptación / Validación	● Se detecta baja demanda. ● Se reduce número de trenes/vagones. ● Ajustes reflejados en dashboard.				

Tabla 8. Tabla de descripción textual. RF-7 Interfaz de monitoreo en tiempo real.

REQUERIMIENTOS FUNCIONALES					
RF-7 Interfaz de monitoreo en tiempo real	RF-7	Necesidad relacionada	Prioridad 10/10	Interfaz de monitoreo en tiempo real	Visualización en dashboard.
Descripción	El sistema debe mostrar video en vivo, métricas y alertas en un dashboard accesible para operadores.				
Justificación	Facilita la supervisión continua y la toma de decisiones rápidas.				
Origen (Interesado)	Cliente (Operador).				
Criterio de aceptación / Validación	● Video en vivo disponible. ● Métricas visibles en interfaz. ● Alertas generadas en tiempo real.				

Tabla 9. Tabla de descripción textual. RF-8 Visualización gráfica de datos.

REQUERIMIENTOS FUNCIONALES					
RF-8 Visualización gráfica de datos	RF-8	Necesidad relacionada	Prioridad 8/10	Visualización gráfica de datos	Visualización gráfica de métricas.
Descripción	El sistema debe presentar métricas mediante gráficos interactivos en el frontend.				
Justificación	Mejora la comprensión de la información para analistas y operadores.				
Origen (Interesado)	Cliente (Analista).				
Criterio de aceptación / Validación	• Gráficos interactivos disponibles. • Actualización en tiempo real. • Funciones de zoom y filtros.				

Tabla 10. Tabla de descripción textual. RF-9 Monitoreo simultáneo de múltiples cámaras.

REQUERIMIENTOS FUNCIONALES					
RF-9 Monitoreo simultáneo de múltiples cámaras	RF-17	Necesidad relacionada	Prioridad 9/10	Monitoreo simultáneo de múltiples cámaras	Monitoreo simultáneo de cámaras.
Descripción	El sistema debe permitir visualizar varias cámaras en paralelo con baja latencia.				
Justificación	Asegura cobertura completa de estaciones críticas.				
Origen (Interesado)	Cliente (Operador).				
Criterio de aceptación / Validación	• Visualización simultánea de múltiples cámaras. • Latencia $\leq 1s$. • Flujo estable sin pérdida de frames.				

Tabla 11. Tabla de descripción textual. RF-10 Generación de alertas automáticas.

REQUERIMIENTOS FUNCIONALES					
RF-10 Generación de alertas automáticas	RF-10	Necesidad relacionada	Prioridad 10/10	Generación de alertas automáticas	Monitoreo simultáneo de cámaras.
Descripción	El sistema debe notificar a operadores cuando se detecten aglomeraciones o sobrecargas.				
Justificación	Mejora la seguridad y la respuesta operativa.				
Origen (Interesado)	Cliente (Operador).				
Criterio de aceptación / Validación	• Se detectan aglomeraciones/sobrecargas. • Notificación automática en tiempo real. • Alertas diferenciadas por criticidad.				

Tabla 12. Tabla de descripción textual. RF-11 Control de acceso al sistema.

REQUERIMIENTOS FUNCIONALES					
RF-11 Control de acceso al sistema	RF-11	Necesidad relacionada	Prioridad 10/10	Control de acceso al sistema	Autenticación y autorización.
Descripción	El sistema debe permitir autenticación y autorización diferenciada por roles.				
Justificación	Garantiza seguridad y uso controlado de funcionalidades.				
Origen (Interesado)	Cliente (Administrador).				
Criterio de aceptación / Validación	• Autenticación obligatoria. • Roles diferenciados (admin, operador, invitado). • Restricción de funciones críticas.				

Tabla 13. Tabla de descripción textual. RF-12 Dashboard de optimización ferroviaria.

REQUERIMIENTOS FUNCIONALES					
RF-12 Dashboard de optimización ferroviaria	RF-12	Necesidad relacionada	Prioridad 9/10	Dashboard de optimización ferroviaria	Visualización de optimización..
Descripción	El sistema debe mostrar resultados de asignación óptima de trenes en la interfaz.				
Justificación	Permite a los operadores tomar decisiones basadas en datos.				
Origen (Interesado)	Cliente (Operador).				
Criterio de aceptación / Validación	- Resultados visibles en dashboard. - Información actualizada en tiempo real. - Acceso desde navegador web.				

Tabla 14. Tabla de descripción textual. RF-13 Monitoreo de estado del sistema.

REQUERIMIENTOS FUNCIONALES					
RF-13 Monitoreo de estado del sistema	RF-13	Necesidad relacionada	Prioridad 8/10	Monitoreo de estado del sistema	Monitoreo de estado.
Descripción	El sistema debe indicar cámaras activas, objetos detectados y última actualización.				
Justificación	Asegura transparencia y control operativo.				
Origen (Interesado)	Cliente (Operador).				
Criterio de aceptación / Validación	- Estado de cámaras mostrado. - Objetos detectados visibles. - Última actualización registrada.				

Tabla 15. Tabla de descripción textual. RF-14 Generación de reportes históricos.

REQUERIMIENTOS FUNCIONALES					
RF-14 Generación de reportes históricos	RF-14	Necesidad relacionada	Prioridad 8/10	Generación de reportes históricos	Generación de reportes.
Descripción	El sistema debe permitir acceso a datos pasados de ocupación y flujo, exportables en formatos estándar.				
Justificación	Facilita análisis y planificación futura.				
Origen (Interesado)	Cliente (Analista).				
Criterio de aceptación / Validación	• Datos históricos almacenados. • Reportes exportables (PDF/CSV). • Acceso desde interfaz de usuario.				

2.3. Requerimientos No Funcionales.

El proyecto tiene una serie de requerimientos no funcionales explicados a continuación

RNF-1 Escalabilidad por microservicios. El sistema debe estar basado en microservicios independientes para permitir escalabilidad modular.

RNF-2 Rendimiento en tiempo real. El sistema debe procesar video en tiempo real con latencia menor a 1 segundo por fotograma y actualizar métricas cada ≤ 5 segundos.

RNF-3 Confiabilidad y disponibilidad. El sistema debe garantizar disponibilidad mínima del 99% en operación continua y tolerancia a fallos en cámaras o conexiones.

RNF-4 Seguridad de datos. El sistema debe implementar encriptación de datos en tránsito y reposo, además de control de acceso por roles.

RNF-5 Compatibilidad. El frontend debe ser accesible desde navegadores modernos (Chrome, Edge, Firefox) y el backend compatible con Linux y Windows.

RNF-6 Usabilidad. La interfaz debe ser intuitiva y permitir visualización clara de métricas y alertas sin necesidad de capacitación extensa.

RNF-7 Mantenibilidad. El sistema debe tener código versionado en repositorios separados y contar con documentación técnica y guías de despliegue.

2.4. Tablas de descripción textual de Requerimientos No Funcionales.

Tabla 16. Tabla de descripción textual. RNF-1 Escalabilidad por microservicios.

REQUERIMIENTOS NO FUNCIONALES					
RNF-1 Escalabilidad por microservicios	RNF-1	Necesidad relacionada	Prioridad 9/10	Caso de uso / Evento relacionado	Despliegue modular de servicios.
Descripción	La arquitectura debe estar basada en microservicios independientes para permitir escalabilidad modular.				
Justificación	Facilita la evolución futura y la integración de nuevas funcionalidades sin afectar el sistema completo.				
Origen (Interesado)	Equipo técnico.				
Criterio de aceptación / Validación	Se pueden añadir nuevas cámaras o estaciones sin afectar el rendimiento global.				

Tabla 17. Tabla de descripción textual. RNF-2 Rendimiento en tiempo real.

REQUERIMIENTOS NO FUNCIONALES					
RNF-2 Rendimiento en tiempo real	RNF-2	Necesidad relacionada	Prioridad 10/10	Caso de uso / Evento relacionado	Despliegue modular de servicios.
Descripción	El sistema debe procesar video en tiempo real con latencia mínima (<1 segundo por fotograma).				
Justificación	Garantiza que las decisiones operativas se basen en datos actualizados.				
Origen (Interesado)	Cliente (Operador).				
Criterio de aceptación / Validación	El dashboard refleja métricas con actualización cada ≤ 5 segundos.				

Tabla 18. Tabla de descripción textual. RNF-3 Confiabilidad y disponibilidad.

REQUERIMIENTOS NO FUNCIONALES					
RNF-3 Confiabilidad y disponibilidad	RNF-3	Necesidad relacionada	Prioridad 10/10	Caso de uso / Evento relacionado	Despliegue modular de servicios.
Descripción	El sistema debe garantizar disponibilidad mínima del 99% en operación continua.				
Justificación	El metro requiere un sistema confiable para no interrumpir el servicio.				
Origen (Interesado)	Cliente (Administrador).				
Criterio de aceptación / Validación	El sistema se mantiene operativo incluso ante fallos de cámaras o conexiones.				

Tabla 19. Tabla de descripción textual. RNF-4 Seguridad de datos.

REQUERIMIENTOS NO FUNCIONALES					
RNF-4 Seguridad de datos	RNF-4	Necesidad relacionada	Prioridad 10/10	Caso de uso / Evento relacionado	Encriptación y control de acceso.
Descripción	El sistema debe implementar encriptación de datos en tránsito y en reposo, además de control de acceso por roles.				
Justificación	Protege la información sensible de usuarios y operaciones.				
Origen (Interesado)	Cliente (Administrador).				
Criterio de aceptación / Validación	Los datos transmitidos están cifrados y los usuarios acceden solo a funciones permitidas.				

Tabla 20. Tabla de descripción textual. RNF-5 Compatibilidad.

REQUERIMIENTOS NO FUNCIONALES					
RNF-5 Compatibilidad	RNF-5	Necesidad relacionada	Prioridad 7/10	Caso de uso / Evento relacionado	Compatibilidad multiplataforma.
Descripción	El frontend debe ser accesible desde navegadores modernos y el backend compatible con Linux y Windows.				
Justificación	Asegura que el sistema pueda usarse en diferentes entornos.				
Origen (Interesado)	Cliente (Analista).				
Criterio de aceptación / Validación	El sistema funciona correctamente en Chrome, Edge y Firefox, y se despliega en Linux/Windows.				

Tabla 21. Tabla de descripción textual. RNF-6 Usabilidad.

REQUERIMIENTOS NO FUNCIONALES					
RNF-6 Usabilidad	RNF-6	Necesidad relacionada	Prioridad 7/10	Caso de uso / Evento relacionado	Compatibilidad multiplataforma.
Descripción	La interfaz debe ser intuitiva y permitir visualización clara de métricas y alertas.				
Justificación	Mejora la experiencia de operadores y analistas.				
Origen (Interesado)	Cliente (Analista).				
Criterio de aceptación / Validación	El sistema funciona correctamente en Chrome, Edge y Firefox, y se despliega en Linux/Windows.				

Tabla 22. Tabla de descripción textual. RNF-7 Mantenibilidad.

REQUERIMIENTOS NO FUNCIONALES					
RNF-7 Mantenibilidad	RNF-7	Necesidad relacionada	Prioridad 8/10	Caso de uso / Evento relacionado	Gestión modular del código.
Descripción	El código debe estar versionado en repositorios separados y contar con documentación técnica.				
Justificación	Facilita la actualización y corrección de errores				
Origen (Interesado)	Cliente (Operador).				
Criterio de aceptación / Validación	Los repositorios permiten despliegue independiente y están documentados.				

2.5. Planteamiento casos de uso de requerimientos funcionales

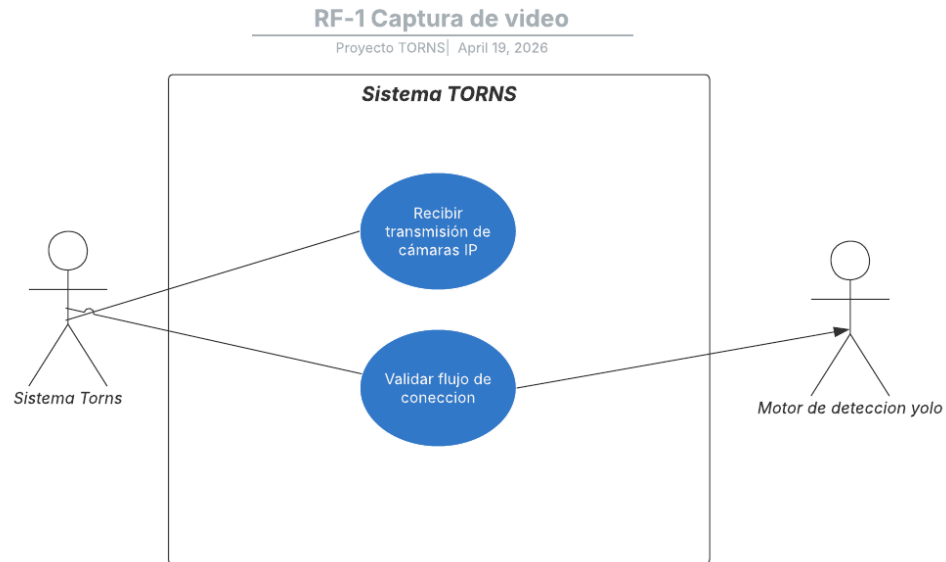
RF-1 Captura de video en estaciones

Caso de uso: Recepción de video en tiempo real

- **Actor:** Sistema TORNNS
- **Propósito:** Recibir transmisión continua desde cámaras IP.
- **Flujo principal:**
 1. La cámara IP envía señal de video.
 2. El sistema recibe y valida el flujo.
 3. El video se transmite al módulo de detección.
- **Resultado esperado:** Video en vivo disponible para análisis.

Diagrama de caso de uso RF-1:

Ilustración 7. RF-1. Fuente propia



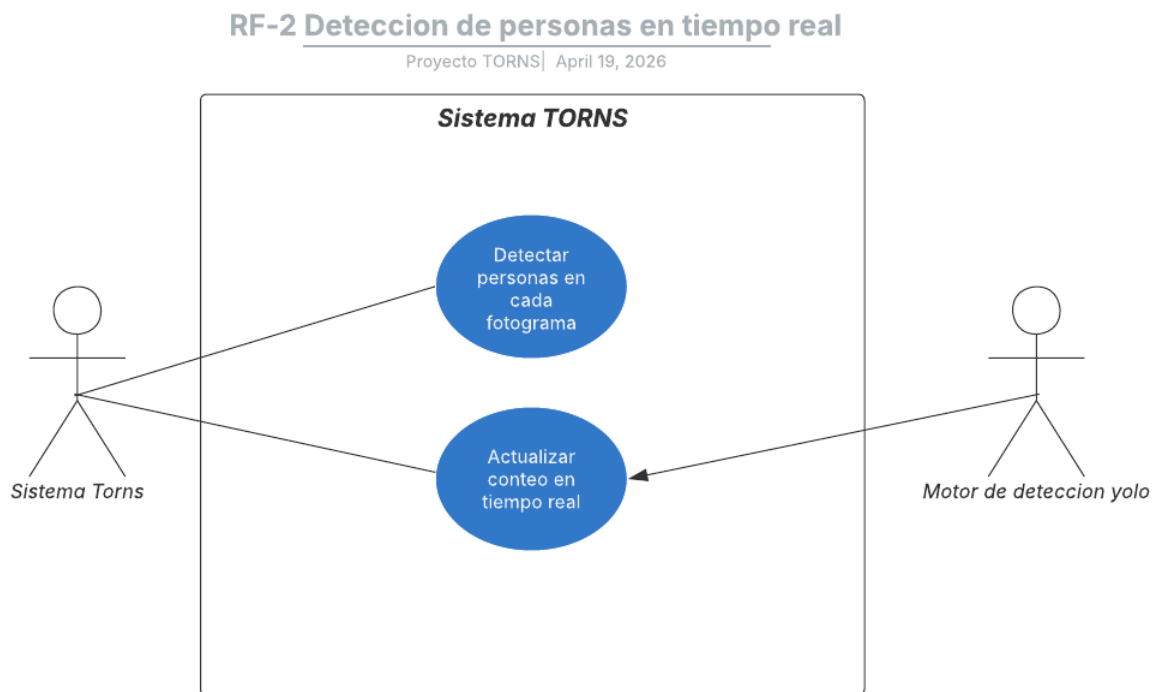
RF-2 Detección de personas en tiempo real

Caso de uso: Conteo de pasajeros por fotograma

- **Actor:** Sistema TORNS
- **Propósito:** Identificar y contabilizar pasajeros mediante YOLO.
- **Flujo principal:**
 1. El sistema procesa cada fotograma.
 2. YOLO detecta personas.
 3. Se actualiza conteo en tiempo real.
- **Resultado esperado:** Conteo $\geq 80\%$ de precisión.

Diagrama de caso de uso RF-2:

Ilustración 8. RF-2. Fuente propia



RF-3 Generación de métricas de ocupación

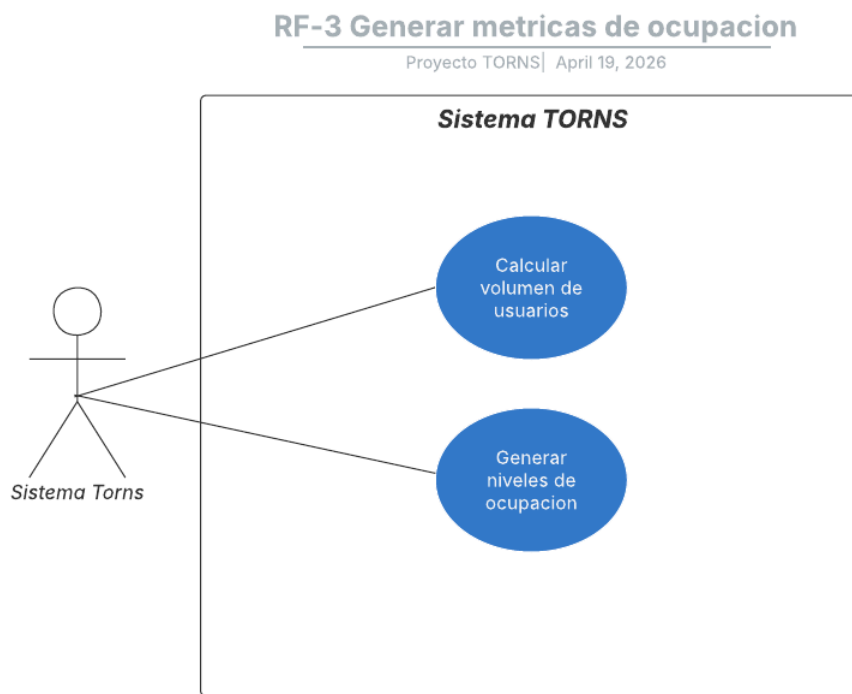
Caso de uso: Cálculo de ocupación por estación

- **Actor:** Sistema TORNS
- **Propósito:** Generar métricas de volumen y niveles de ocupación.
- **Flujo principal:**
 1. Se recibe conteo de pasajeros.
 2. Se calcula volumen por estación.

3. Se asigna nivel (bajo, medio, alto).
- **Resultado esperado:** Métricas actualizadas ≤ 1 min.

Diagrama de caso de uso RF-3:

Ilustración 9. RF-3. Fuente propia



RF-4 Optimización de asignación de trenes y vagones

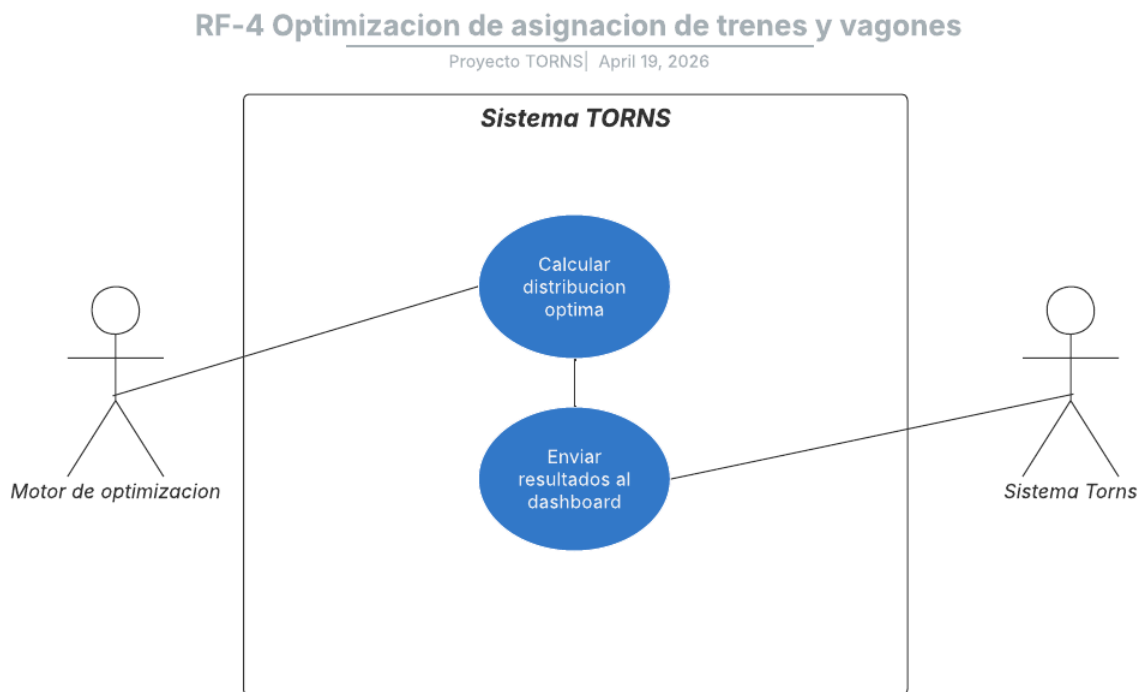
Caso de uso: Distribución óptima de recursos ferroviarios

- **Actor:** Motor de optimización
- **Propósito:** Calcular asignación óptima según demanda.
- **Flujo principal:**

1. Se reciben métricas de ocupación.
 2. Algoritmo calcula distribución óptima.
 3. Resultados se envían al dashboard.
- **Resultado esperado:** Asignación óptima reflejada en interfaz.

Diagrama de caso de uso RF-4:

Ilustración 10. RF-3. Fuente propia



RF-5 Redistribución en horas pico

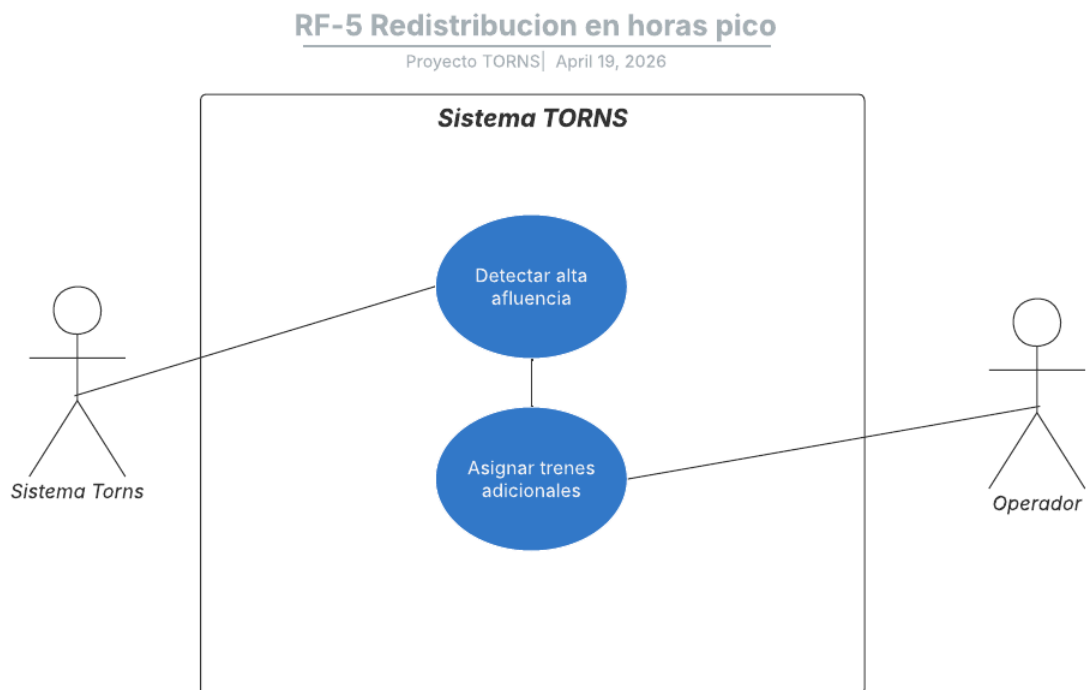
Caso de uso: Asignación dinámica en alta demanda

- **Actor:** Sistema TORNIS

- **Propósito:** Detectar afluencia y asignar trenes adicionales.
- **Flujo principal:**
 1. Se detecta alta ocupación.
 2. Se calculan trenes adicionales.
 3. Dashboard refleja cambios.
- **Resultado esperado:** Reducción de congestión en horas pico.

Diagrama de caso de uso RF-5:

Ilustración 11. RF-5. Fuente propia



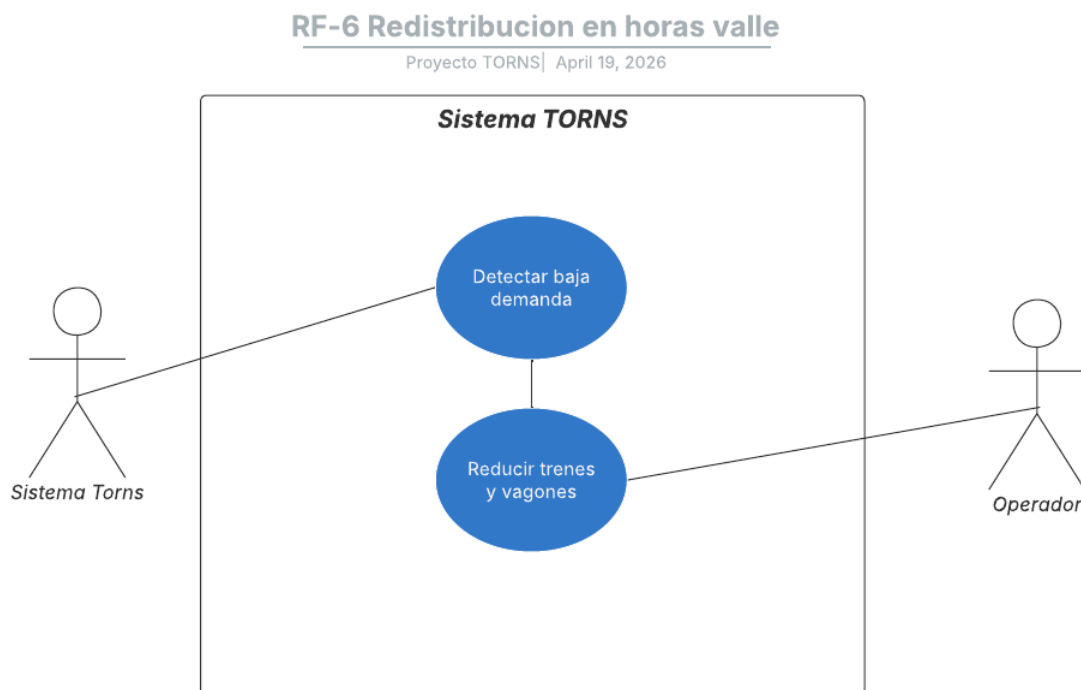
RF-6 Reducción de recursos en horas valle

Caso de uso: Ajuste dinámico en baja demanda

- **Actor:** Sistema TORNS
- **Propósito:** Reducir trenes y vagones en periodos valle.
- **Flujo principal:**
 1. Se detecta baja ocupación.
 2. Se disminuye número de trenes/vagones.
 3. Dashboard refleja cambios.
- **Resultado esperado:** Optimización de costos operativos.

Diagrama de caso de uso RF-6:

Ilustración 12. RF-6. Fuente propia



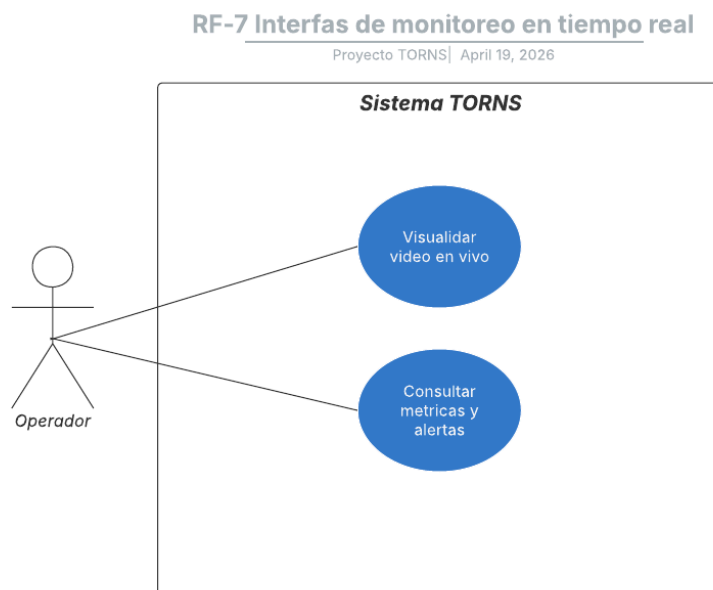
RF-7 Interfaz de monitoreo en tiempo real

Caso de uso: Visualización de video y métricas

- **Actor:** Operador
- **Propósito:** Monitorear video, métricas y alertas en dashboard.
- **Flujo principal:**
 1. Operador accede al dashboard.
 2. Se muestran video en vivo y métricas.
 3. Se generan alertas en tiempo real.
- **Resultado esperado:** Operador informado en todo momento.

Diagrama de caso de uso RF-7:

Ilustración 13. RF-7. Fuente propia



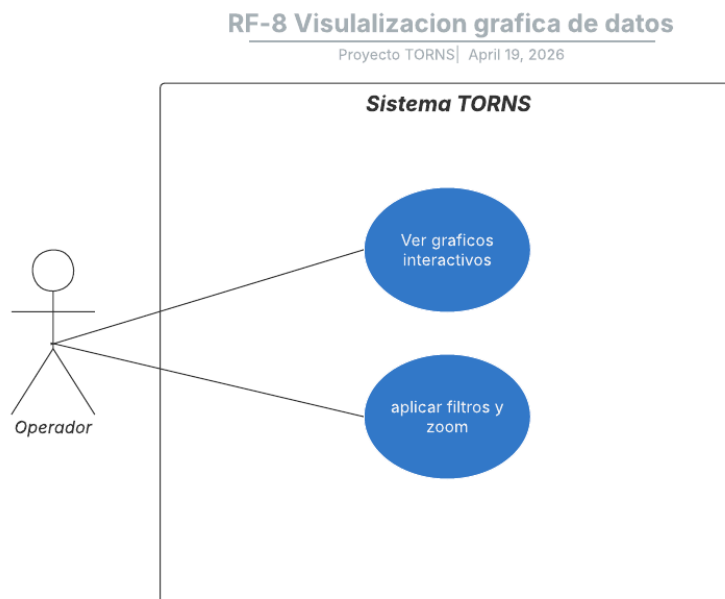
RF-8 Visualización gráfica de los datos

Caso de uso: Gráficos interactivos de ocupación

- **Actor:** Operador
- **Propósito:** Consultar métricas en gráficos interactivos.
- **Flujo principal:**
 1. Operador accede al frontend.
 2. Se muestran gráficos interactivos.
 3. Se aplican filtros o zoom.
- **Resultado esperado:** Visualización clara y manipulable.

Diagrama de caso de uso RF-8:

Ilustración 14. RF-8. Fuente propia



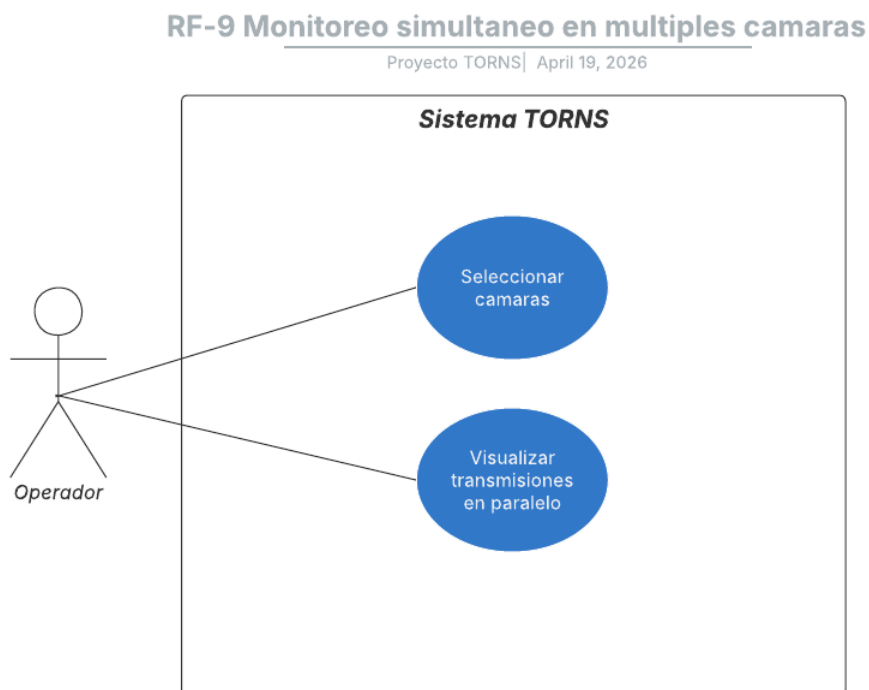
RF-9 Monitoreo simultáneo en múltiples cámaras

Caso de uso: Visualización paralela de cámaras

- **Actor:** Operador
- **Propósito:** Monitorear varias cámaras con baja latencia.
- **Flujo principal:**
 1. Operador selecciona cámaras.
 2. Sistema muestra transmisiones en paralelo.
 3. Flujo se mantiene estable.
- **Resultado esperado:** Monitoreo simultáneo sin pérdida de frames.

Diagrama de caso de uso RF-9:

Ilustración 15. RF-9. Fuente propia

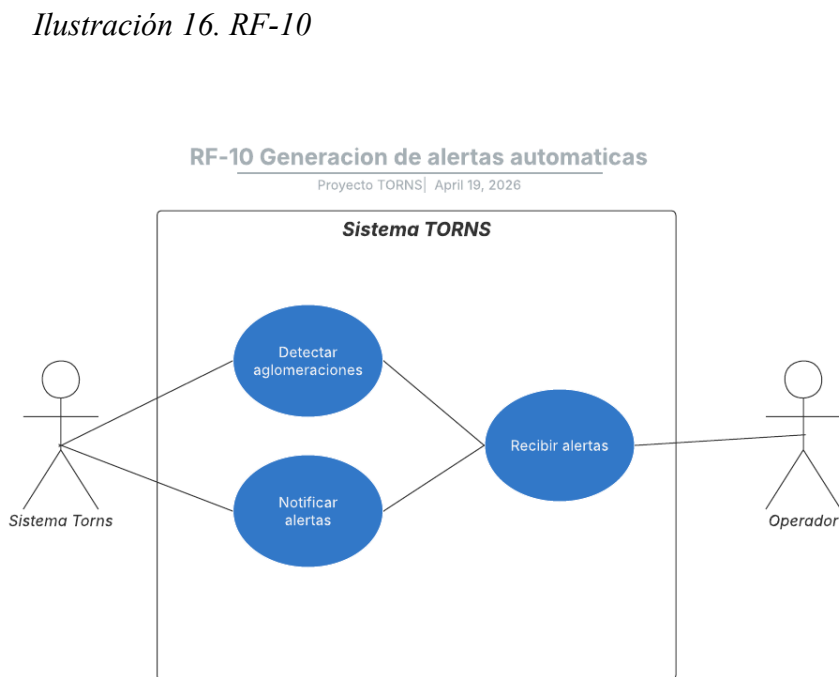


RF-10 Generación de alertas automáticas

Caso de uso: Notificación de aglomeraciones

- **Actor:** Sistema TORNS
- **Propósito:** Notificar operadores ante sobrecargas.
- **Flujo principal:**
 1. Se detecta aglomeración.
 2. Se genera alerta automática.
 3. Operador recibe notificación.
- **Resultado esperado:** Respuesta rápida ante riesgos.

Diagrama de caso de uso RF-10:



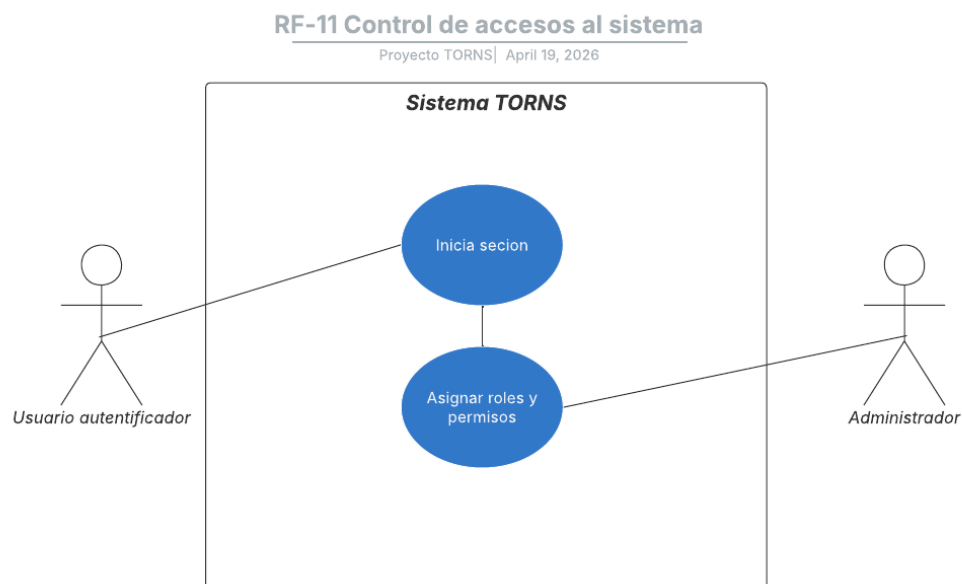
RF-11 Control de accesos al sistema

Caso de uso: Autenticación y autorización por roles

- **Actor:** Usuario (admin, operador, invitado)
- **Propósito:** Acceder al sistema según rol.
- **Flujo principal:**
 1. Usuario ingresa credenciales.
 2. Sistema valida autenticación.
 3. Se asignan permisos según rol.
- **Resultado esperado:** Acceso seguro y diferenciado.

Diagrama de caso de uso RF-11:

Ilustración 17. RF-11. Fuente propia



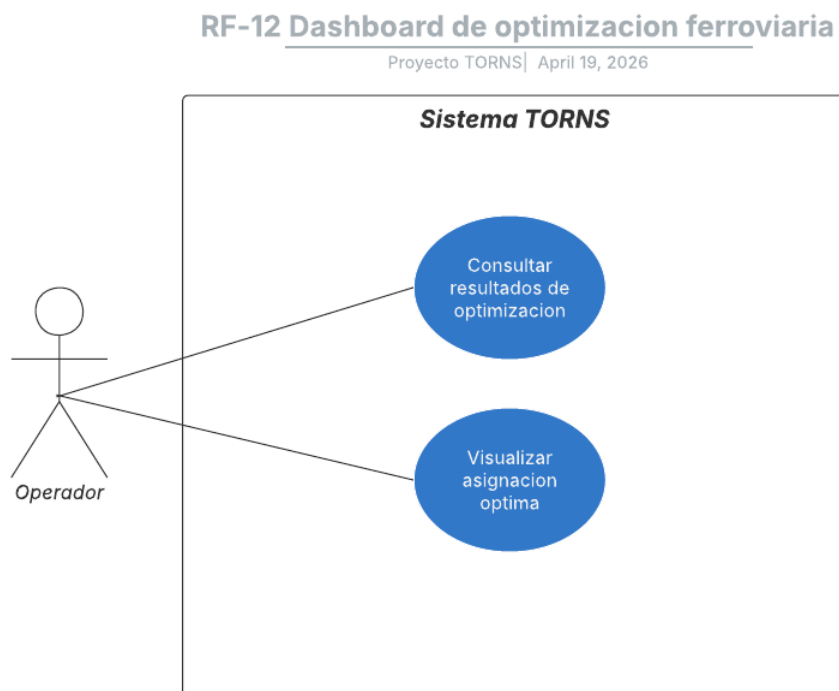
RF-12 Dashboard de optimización ferroviaria

Caso de uso: Visualización de resultados de optimización

- **Actor:** Operador
- **Propósito:** Consultar asignación óptima de trenes.
- **Flujo principal:**
 1. Operador accede al dashboard.
 2. Se muestran resultados de optimización.
 3. Información se actualiza en tiempo real.
- **Resultado esperado:** Operador toma decisiones informadas.

Diagrama de caso de uso RF-12:

Ilustración 18. RF-12. Fuente propia



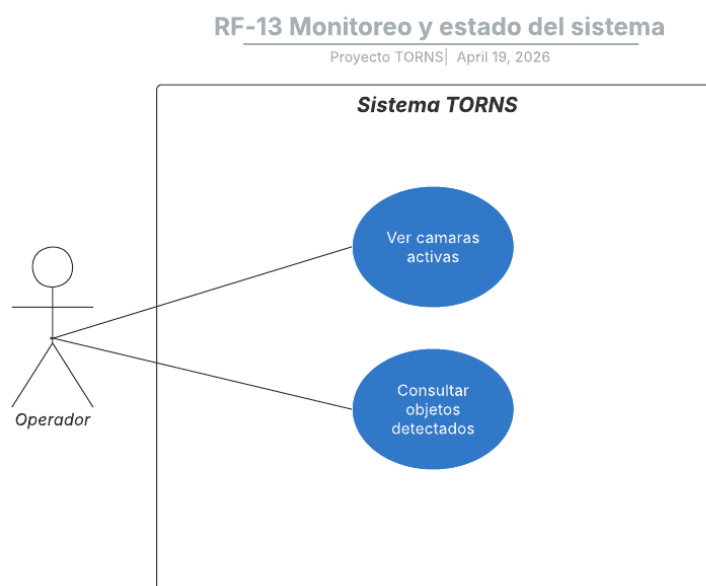
RF-13 Monitoreo y estado del sistema

Caso de uso: Estado de cámaras y detecciones

- **Actor:** Operador
- **Propósito:** Consultar estado del sistema en tiempo real.
- **Flujo principal:**
 1. Operador accede al panel de estado.
 2. Se muestran cámaras activas y objetos detectados.
 3. Se registra última actualización.
- **Resultado esperado:** Información confiable del sistema.

Diagrama de caso de uso RF-13:

Ilustración 19. RF-13. Fuente propia

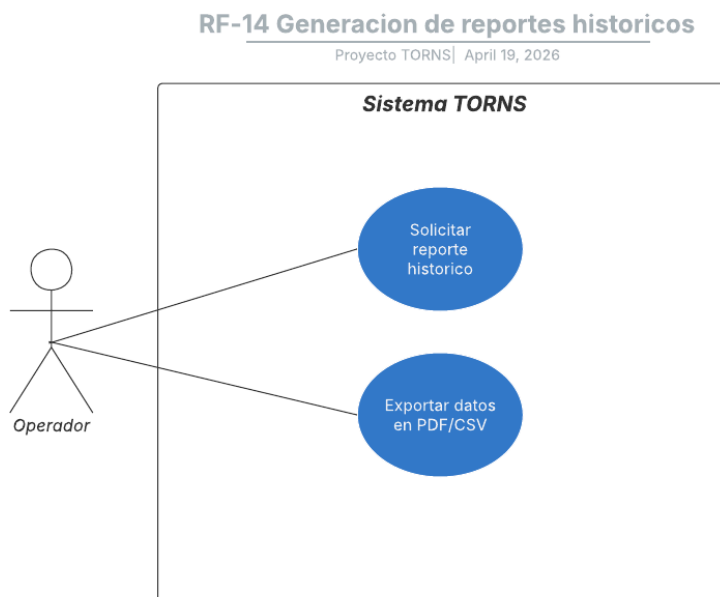


RF-14 Generación de reportes históricos

Caso de uso: Consulta de datos pasados

- **Actor:** Operador
- **Propósito:** Acceder a reportes históricos de ocupación y flujo.
- **Flujo principal:**
 1. Operador solicita reporte.
 2. Sistema genera datos históricos.
 3. Reporte exportable en PDF/CSV.
- **Resultado esperado:** Información histórica disponible para análisis.

Diagrama de caso de uso RF-14:



2.6. Planteamiento casos de uso de no requerimientos funcionales

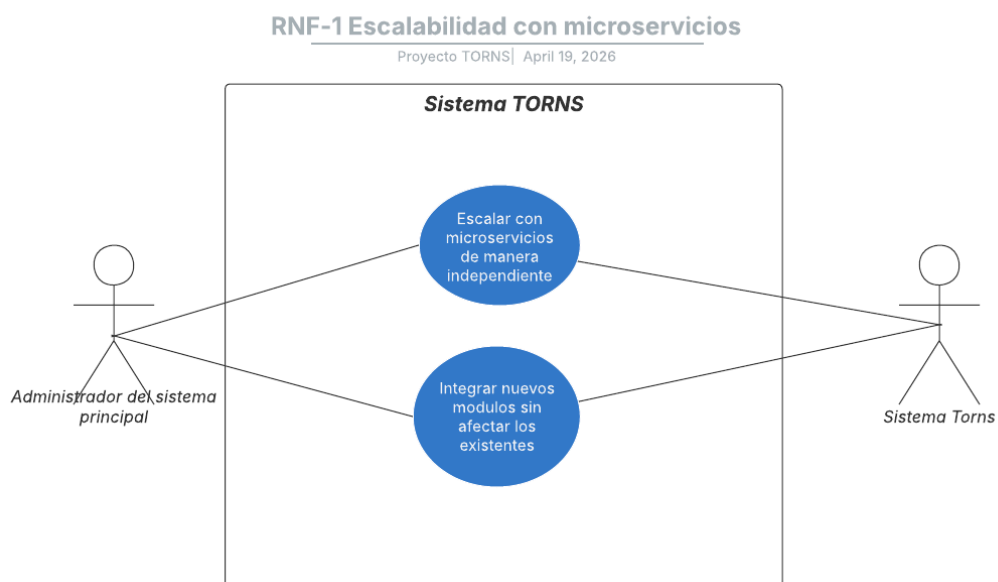
RNF-1 Escalabilidad por microservicios

Caso de uso: Escalado modular del sistema

- **Actor:** Administrador del sistema
- **Propósito:** Escalar módulos de manera independiente.
- **Flujo principal:**
 1. Administrador despliega un nuevo microservicio.
 2. El sistema integra el servicio sin afectar los demás.
 3. Se valida que el sistema continúe operativo.
- **Resultado esperado:** Escalabilidad modular garantizada.

Diagrama de caso de uso RNF-1:

Ilustración 21. RNF-1. Fuente propia



RNF-2 Rendimiento en tiempo real

Caso de uso: Procesamiento de video con baja latencia

- **Actor:** Sistema TORNs
- **Propósito:** Procesar video en tiempo real con latencia $<1s$.
- **Flujo principal:**
 1. El sistema recibe flujo de video.
 2. Procesa cada fotograma en menos de 1s.
 3. Actualiza métricas cada $\leq 5s$.
- **Resultado esperado:** Rendimiento en tiempo real validado.

Diagrama de caso de uso RNF-2:

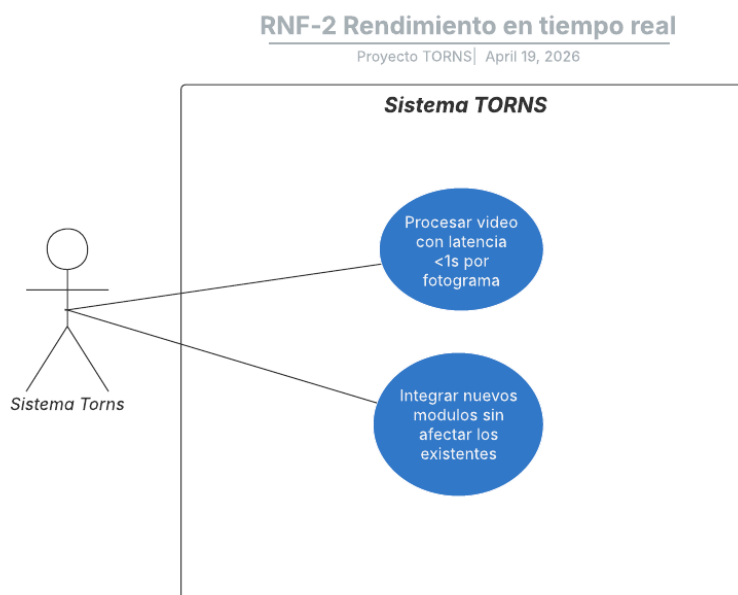


Ilustración 22. RNF-2. Fuente propia

RNF-3 Confiabilidad y disponibilidad

Caso de uso: Operación continua con tolerancia a fallos

- **Actor:** Sistema TORNs
- **Propósito:** Garantizar disponibilidad $\geq 99\%$.
- **Flujo principal:**
 1. El sistema opera en condiciones normales.
 2. Se simula fallo en cámara o conexión.
 3. El sistema mantiene operación con redundancia.
- **Resultado esperado:** Disponibilidad mínima del 99% asegurada.

Diagrama de caso de uso RNF-3:

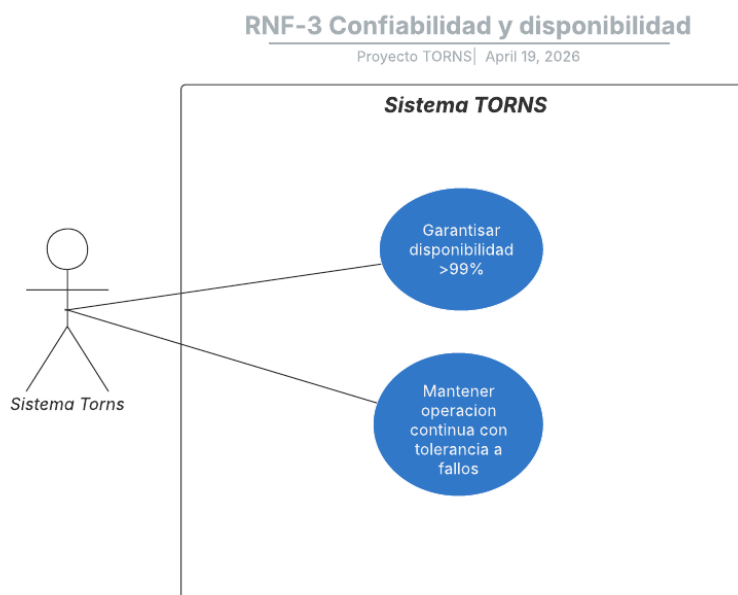


Ilustración 23. RNF-3. Fuente propia

RNF-4 Seguridad de datos

Caso de uso: Protección de datos en tránsito y reposo

- **Actor:** Usuario autenticado
- **Propósito:** Acceder al sistema con datos seguros.
- **Flujo principal:**
 1. Usuario inicia sesión.
 2. Datos se transmiten encriptados.
 3. Acceso se controla por roles.
- **Resultado esperado:** Seguridad de datos garantizada.

Diagrama de caso de uso RNF-4:

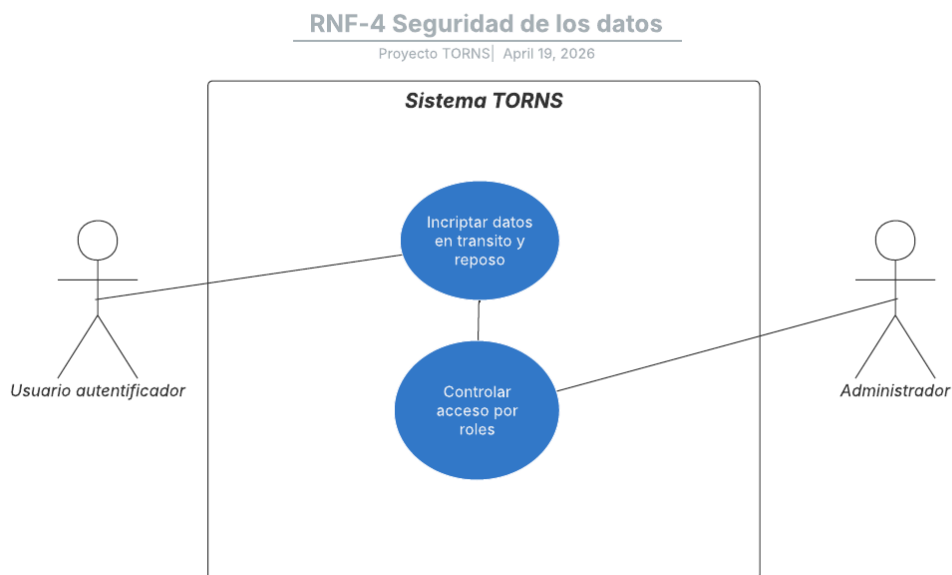


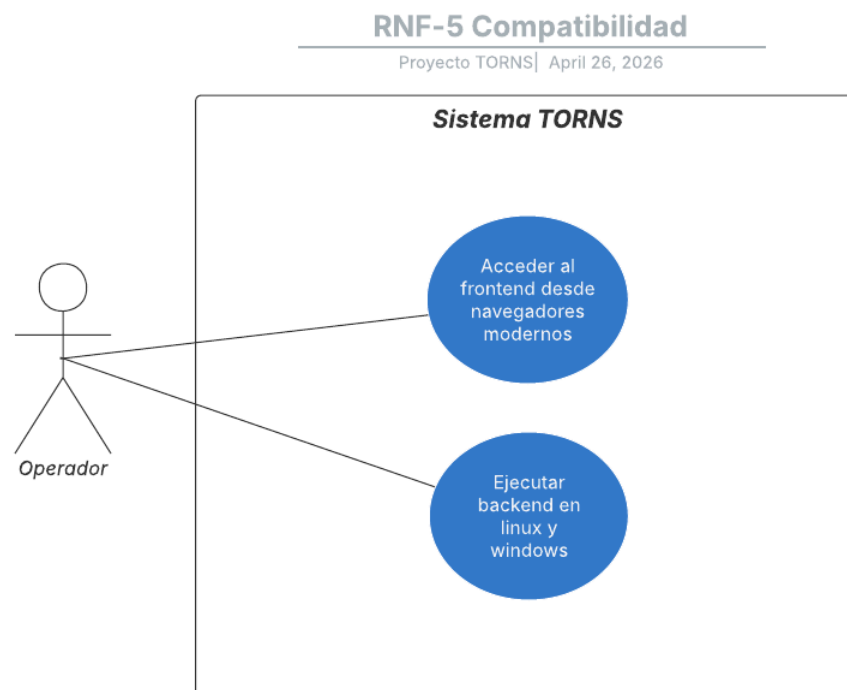
Ilustración 24. RNF-4. Fuente propia

RNF-5 Compatibilidad

Caso de uso: Acceso multiplataforma

- **Actor:** Operador
- **Propósito:** Acceder al sistema desde distintos navegadores y SO.
- **Flujo principal:**
 1. Operador abre frontend en Chrome/Edge/Firefox.
 2. Backend se ejecuta en Linux o Windows.
 3. Se valida compatibilidad en ambos entornos.
- **Resultado esperado:** Acceso multiplataforma sin errores.

Diagrama de caso de uso RNF-5:



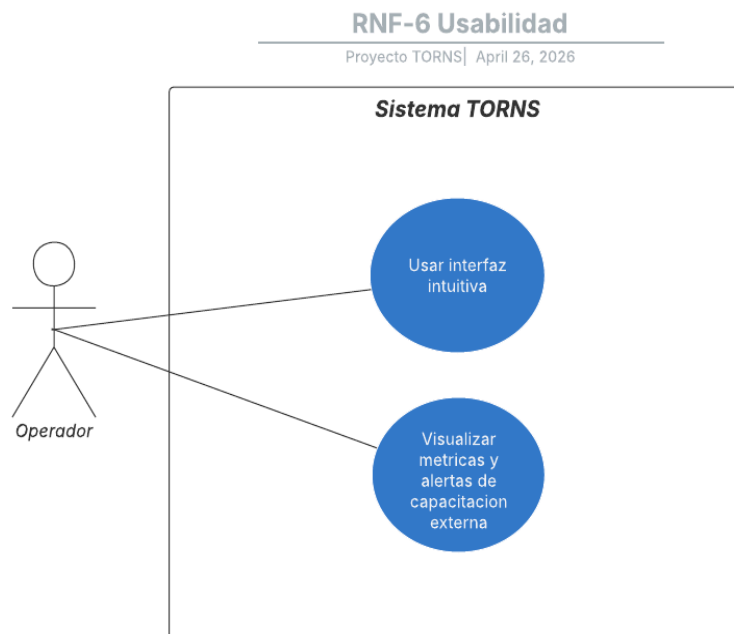
RNF-6 Usabilidad

Caso de uso: Interfaz intuitiva para operadores

- **Actor:** Operador
- **Propósito:** Visualizar métricas y alertas sin capacitación extensa.
- **Flujo principal:**
 1. Operador accede al dashboard.
 2. Interpreta métricas y alertas fácilmente.
 3. Realiza acciones sin necesidad de entrenamiento adicional.
- **Resultado esperado:** Interfaz usable e intuitiva.

Diagrama de caso de uso RNF-6:

Ilustración 26. RNF-6. Fuente propia



RNF-7 Mantenibilidad

Caso de uso: Gestión de código y documentación

- **Actor:** Equipo de desarrollo
- **Propósito:** Mantener código versionado y documentado.
- **Flujo principal:**
 1. Desarrollador actualiza código en repositorio.
 2. Se genera documentación técnica.
 3. Se despliega el sistema siguiendo guías.
- **Resultado esperado:** Sistema mantenible y bien documentado.

Diagrama de caso de uso RNF-7:

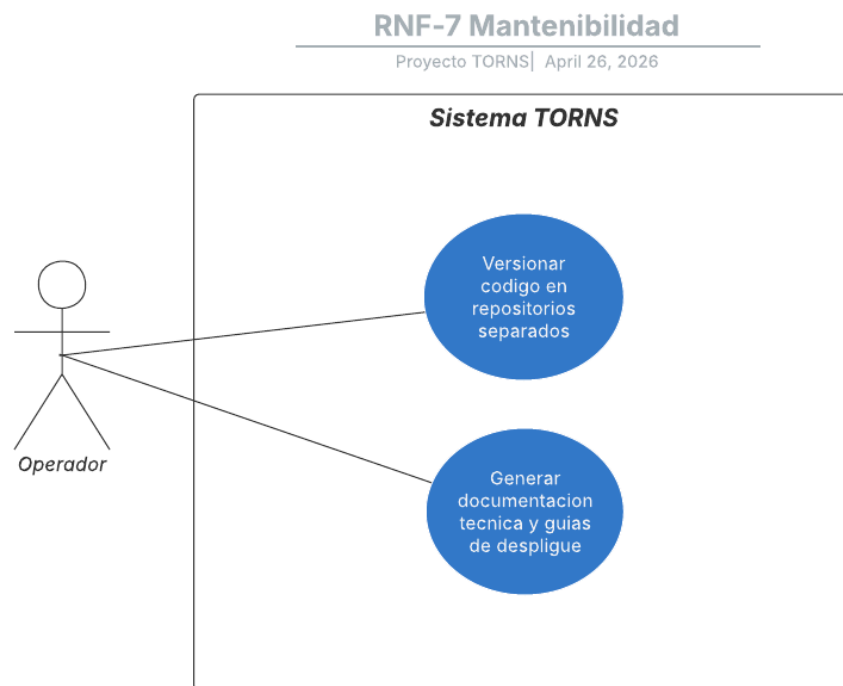


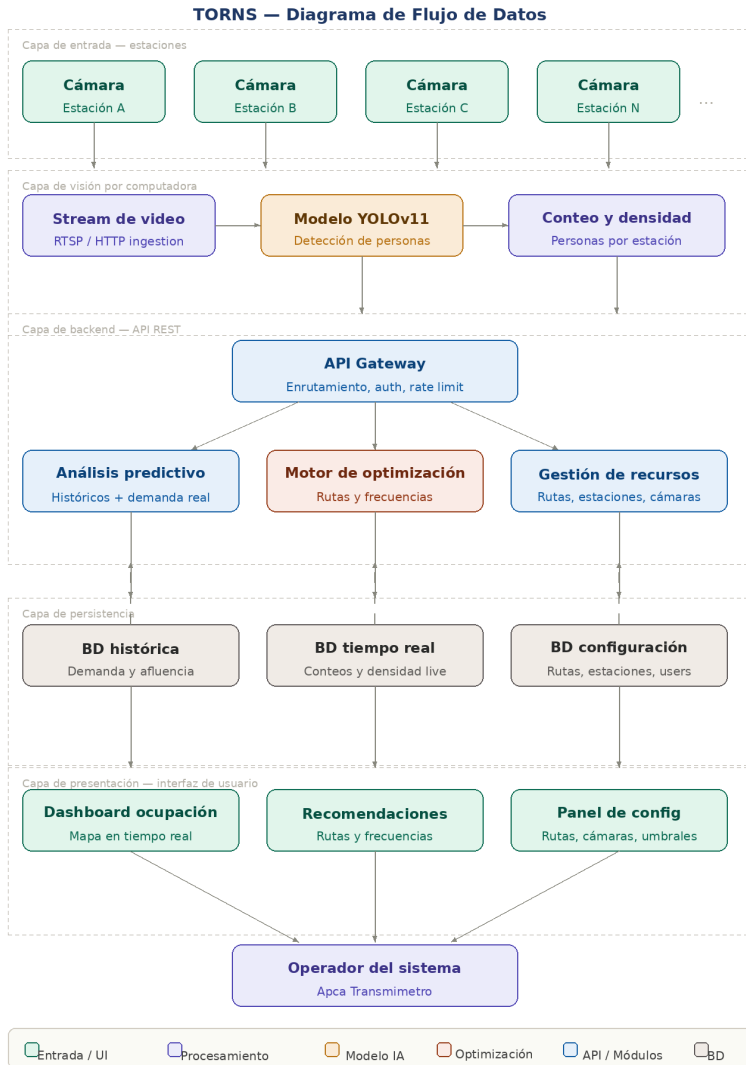
Ilustración 27. RNF-7. Fuente propia

Capítulo 3. Diagramación de software

3.1. Diagrama de Flujo de Datos.

A continuación se puede observar el diagrama de flujo de datos del sistema. En el cual se evidencia el flujo de recorrido de toda la información del proyecto, y la identificación de módulos principales.

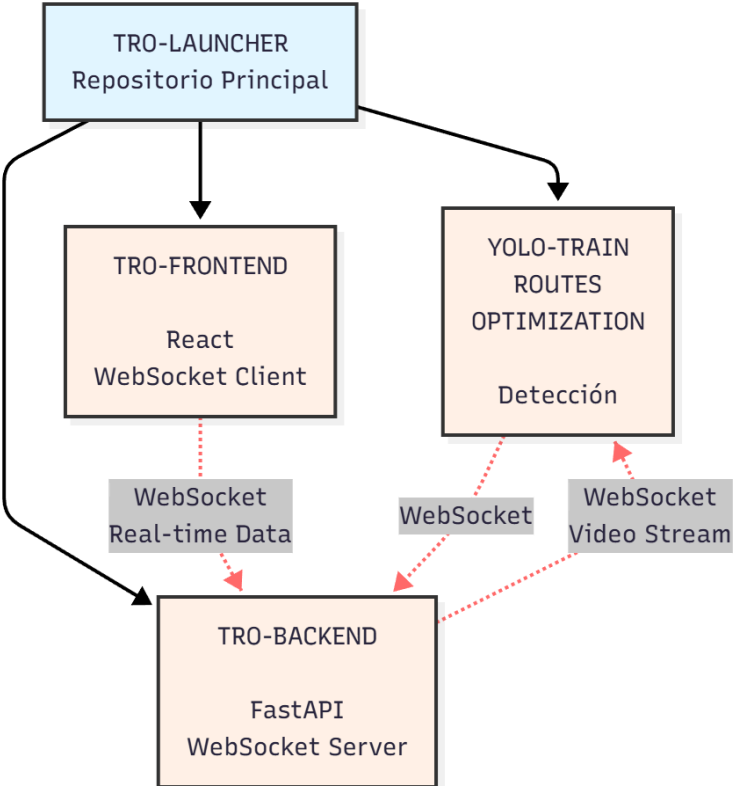
Ilustración 28. Diagrama de Flujo de Datos. Fuente propia



3.2. Diagrama de Arquitectura.

A continuación se puede observar el diagrama de arquitectura del sistema. El detalle específico de toda la arquitectura se puede encontrar en el capítulo de Arquitectura del sistema.

Ilustración 29. Diagrama de Arquitectura Sistema TORNs. Fuente propia

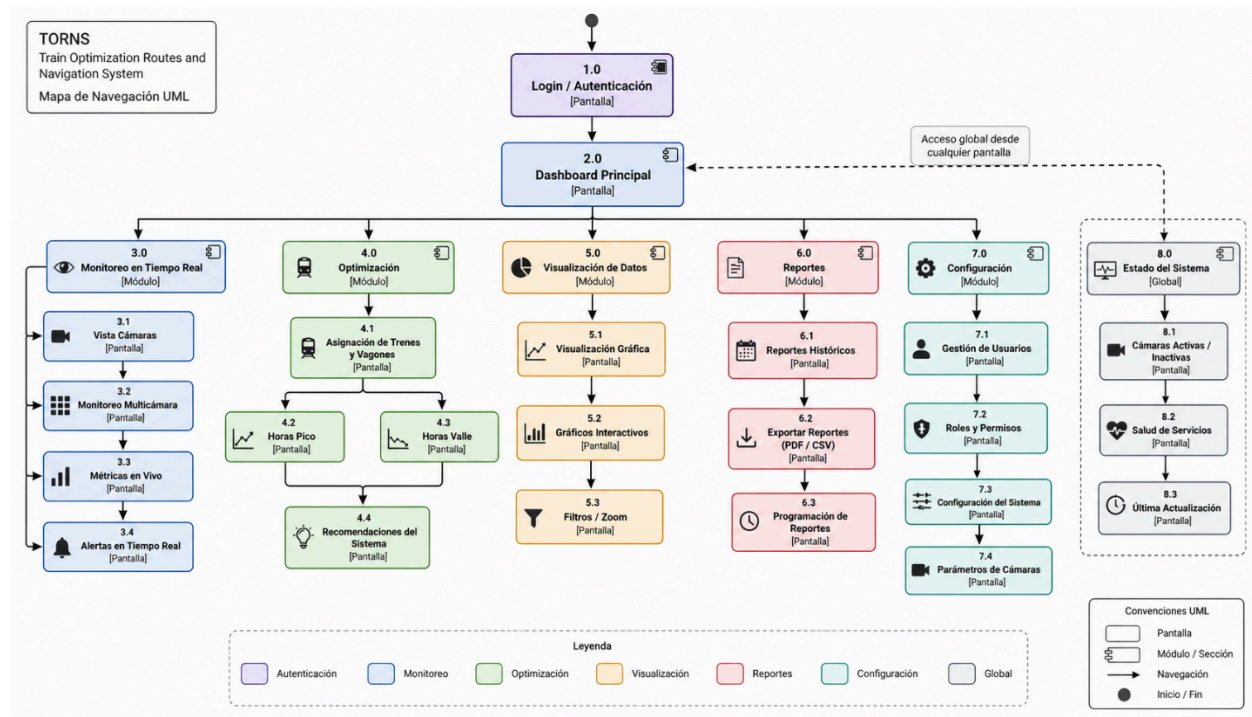


Capítulo 4. Diseño de Interfaz de Usuario

4.1. Mapa de navegación.

A continuación se puede observar el mapa de navegación del sistema. El detalle específico de toda la navegación se puede encontrar en el capítulo diseño de interfaz de usuario.

Ilustración 30. Mapa de Navegación



4.2. Mockups.

En este apartado se expondran los mokups del proyecto mostrando un diseño de alta y baja fidelidad:

Es posible acceder a los mockups a traves del siguiente enlace.

Rebeca Pedrozo Cueto. (s. f.). *Figma - TORNS - MOCKUPS. Figma.*

<https://gadget-agate-35431220.figma.site>

4.3. Prototipo funcional:

El prototipo interactivo del sistema TRO (Train Routes Optimization) fue desarrollado usando Figma Make, una poderosa herramienta de prototipado con inteligencia artificial. El objetivo era crear una experiencia completamente interactiva y fiel al sistema real, basándose en el código existente del frontend (React + TypeScript) y el backend (FastAPI + Python).

El prototipo cuenta con 4 vistas o páginas principales, reflejando la estructura de navegación del frontend real:

1. Home (/): Página de inicio con resumen de estado del sistema
2. Monitoreo (/monitoring): Vista en tiempo real de cámaras y detecciones
3. Cámaras (/cameras): Gestión y configuración de cámaras
4. Rutas (/routing): Optimización de rutas y despacho de trenes

Cada vista se diseñó siguiendo fielmente el layout, estilo visual y composición de componentes definidos en el código React existente.

A continuación se desglosan los principales componentes implementados en cada vista del prototipo:

4.3.1 Home

1. Hero section con título, subtítulo y badge
2. 4 cards con estadísticas clave del sistema (cámaras disponibles, versión, entorno, estado)
3. Botón "Iniciar Monitoreo en Vivo" que navega a </monitoring>
4. Grid de características del sistema (detección inteligente, tiempo real, análisis de flujo, seguridad)

4.3.2 Monitoreo

1. Header con título, chips de estado, e indicadores en tiempo real (conteo de personas, objetos, última actualización)
2. Layout de dos columnas:
3. Sidebar izquierdo con lista de cámaras seleccionables
4. Contenido principal dividido en 3 secciones apiladas verticalmente:
 - 4.1.1.1. Barra superior con información de la cámara activa y stats rápidos
 - 4.1.1.2. Video principal simulando el stream de la cámara con bounding boxes de detección
 - 4.1.1.3. Panel inferior con datos de la estación vinculada a la cámara (pasajeros detectados, último despacho, estado)
5. Estado vacío cuando no hay cámara seleccionada

4.3.3 Cámaras

1. Header con título y subtítulo

2. Grid de tarjetas, una por cada cámara configurada, mostrando:

2.1.1. Ícono, badge de estado y nombre

2.1.2. Metadatos (ID, resolución, FPS)

2.1.3. Stats (tiempo activo)

4.3.4 Cámaras

1. Header con título, subtítulo y botón de actualización

1.1. Tabs horizontales:

1.1.1. Vista General

1.1.1.1. Sección con grid de tarjetas de estado por estación (pasajeros, trenes, estado, última actualización)

1.1.1.2. Sección con 3 cards de resumen (total modelos de tren, estaciones configuradas, estaciones activas)

1.1.2. Modelos de Tren

1.1.2.1. Grid de tarjetas por modelo (nombre, vagones, capacidad)

1.1.3. Estaciones

1.1.3.1. Grid de tarjetas por estación (nombre, línea, cámara vinculada, tiempo de viaje, trenes soportados)

1.1.4. Calculadora de Despacho

1.1.4.1. Formulario con selector de estación, input de pasajeros, checkbox de persistencia y botones (submit/reset)

1.1.4.2. Panel de resultado del plan de despacho calculado

4.3.5 Flujos de Interacción

Todos los flujos de interacción clave entre estas vistas y componentes fueron implementados en el prototipo, incluyendo:

1. Navegación por Navbar y tabs
2. Click en "Iniciar Monitoreo" desde Home
3. Selección de cámaras en Monitoreo y actualización del stream y panel de datos
4. Validación y envío del formulario de la Calculadora de Despacho
5. Skeleton screens y estados de loading

Es posible acceder al prototipo funcional a través del siguiente enlace.

Rebeca Pedrozo Cueto. (s. f.). *Figma - TORNS. Figma.*

<https://zone-legend-96465177.figma.site/>

Capítulo 5. Arquitectura del sistema

La arquitectura del sistema se organiza bajo un enfoque distribuido basado en microservicios, el cual permite la separación clara de responsabilidades y la escalabilidad independiente de cada componente. La estructura se compone de tres capas principales. La primera es la capa de presentación, implementada en React.js, encargada de la interacción directa con el usuario, la visualización de métricas, el análisis de datos y la configuración del sistema.

La segunda corresponde a la capa de lógica de negocio, desarrollada con FastAPI, que administra los flujos de datos provenientes del módulo de detección, gestiona la base de datos, ejecuta los algoritmos de optimización y expone los servicios REST. La tercera capa es el módulo de detección basado en YOLO, responsable de procesar continuamente los flujos de video de las estaciones, identificar personas, generar métricas de afluencia y transmitir la información hacia el backend.

Esta organización favorece la respuesta en tiempo real, la separación de responsabilidades y la posibilidad de escalar o mejorar cada módulo de manera independiente.

5.1. YOLO Train Routes Optimization

Es posible acceder al código fuente a través del siguiente enlace.

5arSanti. (s. f.). *GitHub - 5arSanti/yolo-train-routes-optimization*. GitHub.

<https://github.com/5arSanti/yolo-train-routes-optimization>

El módulo YOLO Train Routes Optimization constituye el núcleo de la detección basada en visión por computadora. Este componente emplea modelos YOLO v8 y v11 entrenados para identificar personas en entornos de transporte masivo, operando sobre flujos de video adquiridos en tiempo real desde cámaras IP. Para cumplir con su propósito, el sistema apoya su funcionamiento en OpenCV para el procesamiento de imágenes, PyTorch como framework de aprendizaje profundo y NumPy para el manejo eficiente de operaciones numéricas.

El módulo ofrece capacidades avanzadas como la detección precisa del volumen de personas presentes en las estaciones, la generación automática de conteos y la identificación del movimiento de los usuarios mediante técnicas de tracking entre fotogramas consecutivos.

Asimismo, produce métricas de afluencia que permiten reconocer patrones de densidad, picos de demanda y comportamiento dinámico de los flujos de pasajeros.

5.2. TRO Backend

Es posible acceder al código fuente a través del siguiente enlace.

5arSanti. (s. f.). *GitHub - 5arSanti/tro-backend: Repositorio for Train Routes Optimization Backend Project*. GitHub. <https://github.com/5arSanti/tro-backend>

El backend constituye la capa encargada de coordinar el flujo funcional del sistema. Mediante el uso de FastAPI, SQLAlchemy, Uvicorn y Pydantic, esta componente procesa los datos enviados por el módulo de detección, gestiona la persistencia de la información en la base de datos y ejecuta algoritmos de optimización con base en la demanda registrada.

El backend ofrece endpoints que permiten la consulta de estadísticas, la recepción de datos de detección, la obtención del nivel de ocupación por estación y la emisión de recomendaciones operativas. Además, incorpora métodos especializados para el cálculo del número de vagones requeridos, la frecuencia óptima de trenes en función de la demanda y la predicción de patrones futuros mediante modelos de series temporales.

Este módulo actúa como puente entre el sistema de detección y la capa de visualización, consolidando la información necesaria para la toma de decisiones operativas.

5.3. TRO Frontend

Es posible acceder al código fuente a través del siguiente enlace.

5arSanti. (s. f.). *GitHub - 5arSanti/tro-frontend: Repositorio for Train Roputes Optimization Frontend Project*. GitHub. <https://github.com/5arSanti/tro-frontend>

El frontend constituye la interfaz desde la cual operadores y analistas acceden a la información procesada. Este módulo se desarrolla con React.js y se complementa con herramientas como Vite, Chart.js y Axios para el desarrollo ágil, la visualización gráfica y la comunicación con el backend. El sistema presenta paneles de control en tiempo real que permiten monitorear estaciones, supervisar niveles de ocupación, identificar alertas operativas y acceder a análisis históricos.

También integra funcionalidades para configurar parámetros del sistema, ajustar cámaras, revisar reportes descargables y gestionar rutas operacionales. Esta capa favorece una interacción intuitiva y eficiente, permitiendo que los datos derivados del análisis automático se conviertan en información útil para los operadores del sistema de transporte.

Capítulo 6. Desarrollo - Entrenamiento del modelo

En la fase de entrenamiento del modelo y la la capa de visión por computadora se emplean YOLO como modelo principal de detección, OpenCV para el procesamiento de videos, PyTorch para operaciones de aprendizaje profundo y NumPy para cálculos optimizados.

La arquitectura del modelo esta compuesta por una red neuronal convolucional (CNN) basada en la arquitectura YOLO, específicamente la versión YOLO11s (Optimizada para rapidez y eficiencia).

Esta arquitectura pertenece a la familia de modelos YOLOv11, que siguen la filosofía de detección de objetos en una sola pasada (single-shot detection), donde el modelo predice directamente las coordenadas de los objetos y sus clases en una sola inferencia.

YOLO11s, utiliza internamente una red convolucional profunda para extraer las características visuales de las imágenes.

Está basada en una arquitectura tipo CSP (Cross Stage Partial Network), que mejora la eficiencia dividiendo las capas y reduciendo el cómputo sin perder Red neuronal convolucional (CNN) basada en la arquitectura YOLO, específicamente la versión YOLO11s (Optimizada para rapidez y eficiencia).

Ilustración 31. Configuración del entrenamiento del modelo.

```
clear_folder("runs/detect/train")

model = YOLO("yolo11s.pt")

model.train(
    data="data.yaml",
    model="yolo11s.pt",
    epochs=60,
    imgsz=512,
    batch=16,
    workers=8,
    amp=True,

    # AUGMENTATION - VALIDADO PARA YOLO11
    hsv_h=0.015,      # variación ligera de tono
    hsv_s=0.7,        # saturación fuerte (útil en exteriores)
    hsv_v=0.4,        # variación de brillo moderada
    degrees=5.0,      # rotación suave (personas no deben rotarse mucho)
    translate=0.10,   # desplazamiento leve
    scale=0.50,       # zoom in/out moderado
    shear=1.5,        # deformación suave
    perspective=0.0005, # perspectiva muy ligera
    flipud=0.0,       # NO voltear verticalmente personas
   fliplr=0.5,        # flip horizontal si es natural
    mosaic=1.0,        # mosaico activado (muy útil)
    mixup=0.10,       # mezcla suave
    copy_paste=0.0,   # no recomendado para humanos
    erasing=0.4,      # borrado aleatorio útil, sin exagerar
    crop_fraction=1.0, # 1.0 = sin recorte agresivo
)
```

[illegible]

6.2. Imágenes de validación y predicción.

A continuacion se pueden evidenciar algunas predicciones realizadas con el modelo, en personas reales.

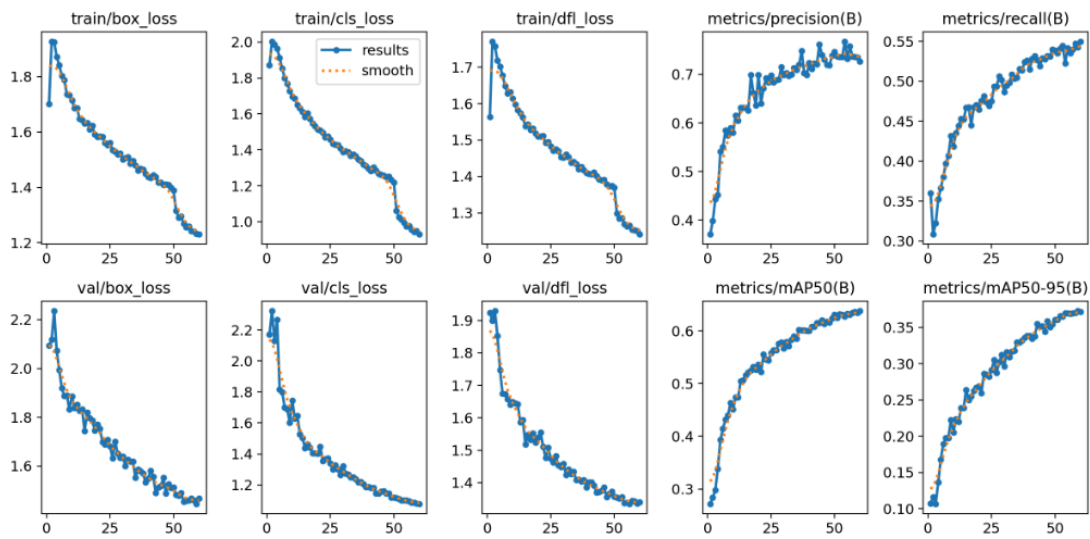
Ilustración 33. Imagenes de validacion y predicción. Fuente: Propia



Durante el entrenamiento del modelo, las métricas registradas muestran un comportamiento óptimo de convergencia y desempeño.

Las pérdidas de entrenamiento y validación (box_loss, cls_loss, dfl_loss) presentan una disminución progresiva y estable a lo largo de las 40 épocas, evidenciando un ajuste efectivo de los parámetros sin signos de sobreajuste.

Ilustración 34. Métricas y curvas de entrenamiento del modelo YOLO a lo largo de las 60 épocas.



Capítulo 7. Base de datos

El sistema TRO (Train Routes Optimization) es una plataforma de monitoreo inteligente que combina detección de personas mediante visión por computadora (modelo YOLO) con un motor de optimización de despacho de trenes en tiempo real. Este documento describe el diseño de la base de datos relacional que soporta dicha plataforma. El sistema opera con los siguientes módulos principales:

1. Módulo de Video y Cámaras: gestión de streams de video de vigilancia.
2. Módulo de Detección IA: procesamiento de frames con YOLO para contar personas y
3. objetos.
4. Módulo de Routing: estaciones, modelos de tren y cálculo de planes de despacho.
5. Módulo de Estado: registro histórico del estado de cada estación y despachos realizados.

Para nuestra base de datos estaremos usando PostgreSQL 15+ como motor de base de datos pues se nos permite dar soporte nativo de tipos JSONB para almacenar métricas de detección variables. Manejo de tipos TIMESTAMPTZ para manejo correcto de zonas horarias en lecturas en tiempo real, tenemos integridad referencial robusta con soporte completo de claves foráneas y finalmente tenemos compatibilidad total con el stack FastAPI + SQLAlchemy del backend existente.

7.1 Resumen de tablas

Tabla 23. Resumen de tablas de la base de datos. Fuente propia.

Tabla	Grupo	Descripción
train_models	Routing	Catálogo de modelos de tren disponibles

stations	Routing	Configuración de estaciones del sistema
station_train_support	Routing	Relación N:M entre estaciones y modelos de tren
cameras	Video	Dispositivos de cámara vinculados a estaciones
detection_events	Detección IA	Eventos de detección de personas por frame
station_states	Estado	Estado operativo actual de cada estación
dispatch_plans	Estado	Histórico de planes de despacho generados

7.2 Esquema relacional

A continuación se describen todas las tablas del sistema con sus columnas, tipos de datos, restricciones y relaciones.

7.2.1 train_models

Catálogo de los modelos de tren disponibles para el sistema. Cada modelo define su configuración física (número de vagones y capacidad por vagón). La capacidad total se puede derivar como columna calculada.

Tabla 24. Resumen de tabla train_models. Fuente propia.

Columna	Tipo	Restricción	Descripción
id	VARCHAR(50)	PK	Identificador único del modelo (ej: 'light', 'medium', 'heavy')
name	VARCHAR(100)	NOT NULL	Nombre descriptivo (ej: 'Tren Ligero (1 vagón)')
wagon_count	SMALLINT	NOT NULL, CHECK ≥ 1	Número de vagones del modelo

capacity_per_wagon	SMALLINT	NOT NULL, CHECK ≥ 1	Pasajeros máximos por vagón
total_capacity	SMALLINT	GENERATED / NOT NULL	Capacidad total = wagon_count $\times$ capacity_per_wagon
total_capacity	SMALLINT	GENERATED / NOT NULL	Capacidad total = wagon_count $\times$ capacity_per_wagon
created_at	TIMESTAMPTZ	NOT NULL, DEFAULT now()	Fecha de registro del modelo

7.2.1.1 Datos semilla del sistema actual:

Tabla 25. Datos semilla. Fuente propia.

Columna	name	wagon_count	capacity_per_wagon	n total_capacity
id	Tren Ligero (1 vagón)	1	5	5
medium	Tren Medio (2 vagones)	2	5	10
heavy	Tren Pesado (3 vagones)	3	5	15

7.2.2 stations

Tabla 26. Resumen de tabla stations. Fuente propia.

Columna	Tipo	Restricción	Descripción
id	VARCHAR(50)	PK	Identificador único de la estación (ej: 'station_central')
name	VARCHAR(100)	NOT NULL	Línea de transporte a la que pertenece
line	VARCHAR(50)	NOT NULL	Línea de transporte a la que pertenece

camera_id	VARCHAR(50)	FK → cameras.id, UNIQUE	Cámara de vigilancia vinculada
travel_time_seconds	INTEGER	NOT NULL, CHECK ≥ 0	Capacidad total = Tiempo promedio de viaje al siguiente punto (segundos)
created_at	TIMESTAMPTZ	NOT NULL, DEFAULT now()	Fecha de registro
updated_at	TIMESTAMPTZ	NOT NULL, DEFAULT now()	Última modificación

7.2.2.1 Datos semilla del sistema actual

Tabla 27. Datos semilla del sistema actual. Fuente propia

id	name	line	camera_id	travel_time_s
station_sur_usb0	Estación Sur USB0	Línea 3	usb0	210
station_sur_usb1	Estación Sur USB1	Línea 3	usb1	210
station_central	Estación Central	Línea 1	video1	180
station_norte	Estación Norte	Línea 2	video2	240
station_east	Estación Este	Línea 4	video3	270
station_west	Estación Oeste	Línea 5)	video4	300

7.2.3 station_train_support

Tabla 28. Resumen de tabla station_train_support. Fuente propia.

Columna	Tipo	Restricción	Descripción
station_id	VARCHAR(50)	PK (compuesta), FK → stations.id	Referencia a la estación
train_model_id	VARCHAR(50)	PK (compuesta), FK → train_models.id	Referencia al modelo de tren
assigned_at	TIMESTAMPTZ	NOT NULL, DEFAULT now()	Fecha en que se asignó el modelo a la estación

7.2.4 cameras

Tabla 29. Resumen de tabla cameras. Fuente propia.

Columna	Tipo	Restricción	Descripción
id	VARCHAR(50)	PK	Identificador del dispositivo (ej: 'video1', 'usb0')
name	VARCHAR(100)	NOT NULL	Nombre descriptivo de la cámara
source_path	VARCHAR(255)	NOT NULL	Ruta al archivo de video o dispositivo
resolution_width	SMALLINT	DEFAULT 1920	Ancho de resolución en píxeles
fps	SMALLINT	DEFAULT 30	Fotogramas por segundo configurados
is_active	BOOLEAN	NOT NULL, DEFAULT TRUE	Indica si la cámara está habilitada
created_at	TIMESTAMPTZ	NOT NULL, DEFAULT now()	Fecha de registro

7.2.5 detection_events

Tabla 30. Resumen de tabla detection_events. Fuente propia.

Columna	Tipo	Restricción	Descripción
id	BIGSERIAL	PK	Identificador autoincremental del evento
camera_id	VARCHAR(50)	NOT NULL, FK → cameras.id	Cámara que generó la detección
person_count	SMALLINT	NOT NULL, CHECK ≥ 0	Número de personas detectadas en el frame
total_objects	SMALLINT	NOT NULL, CHECK ≥ 0	Total de objetos detectados (incluye personas)
avg_confidence	NUMERIC(5,4)	CHECK 0.0–1.0	Confianza promedio del modelo YOLO
frame_timestamp	TIMESTAMPT Z	NOT NULL	Marca de tiempo del frame procesado
raw_detections	JSONB	NULLABLE	Detecciones crudas del modelo (bounding boxes, clases, confianza)
created_at	TIMESTAMPT Z	NOT NULL, DEFAULT now()	Momento de inserción en la BD

7.2.6 station_states

Tabla 31. Resumen de tabla station_states. Fuente propia.

Columna	Tipo	Restricción	Descripción
station_id	VARCHAR(50)	PK, FK → stations.id	Referencia a la estación
last_passenger_count	INTEGER	NULLABLE, CHECK ≥ 0	Último conteo de pasajeros registrado
last_updated	TIMESTAMPTZ	NULLABLE	Momento del último conteo registrado
last_dispatch_id	BIGINT	NULLABLE, FK → dispatch_plans.id	Último plan de despacho generado

7.2.6 dispatch_plans

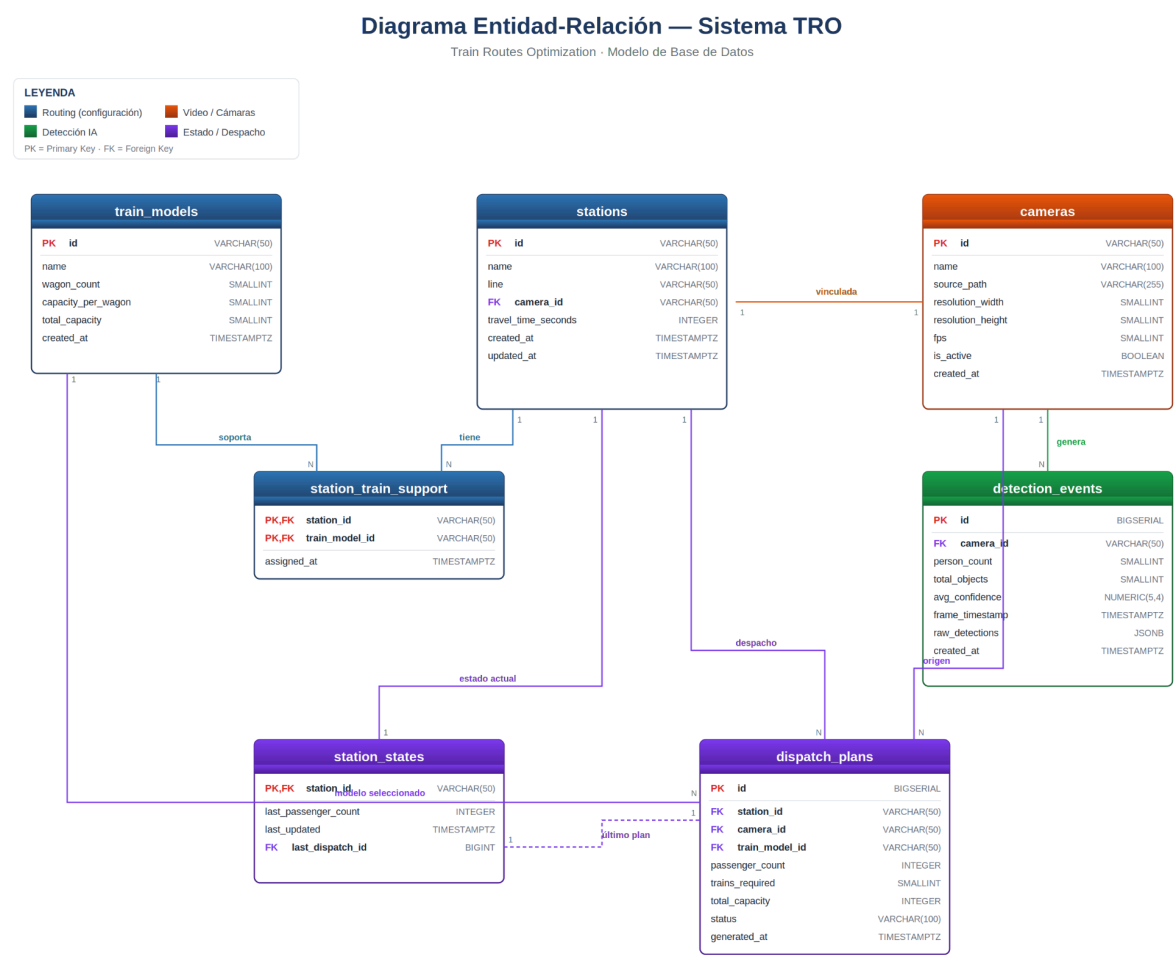
Tabla 32. Resumen de tabla dispatch_plans. Fuente propia.

Columna	Tipo	Restricción	Descripción
id	BIGSERIAL	PK	Identificador del plan de despacho
station_id	VARCHAR(50)	NOT NULL, FK → stations.id	Estación para la que se calculó el plan
camera_id	VARCHAR(50)	NOT NULL, FK → cameras.id	Cámara que originó la lectura de pasajeros
train_model_id	VARCHAR(50)	NOT NULL, FK → train_models.id	Modelo de tren seleccionado por el algoritmo
passenger_count	INTEGER	NOT NULL, CHECK ≥ 0	Número de pasajeros observados al momento del cálculo
trains_required	SMALLINT	NOT NULL, CHECK ≥ 0	Cantidad de trenes recomendados a despachar
total_capacity	INTEGER	NOT NULL, CHECK ≥ 0	Capacidad total ofrecida por los trenes despachados
surplus_capacity	INTEGER	NOT NULL, CHECK ≥ 0	Asientos disponibles sobrantes tras atender la demanda
estimated_cycle_time_seconds	INTEGER	NOT NULL, CHECK ≥ 0	Tiempo estimado del ciclo completo de despacho
status	VARCHAR(100)	NOT NULL	Mensaje descriptivo del resultado (ej: 'Despacho óptimo calculado')
persist	BOOLEAN	NOT NULL, DEFAULT TRUE	Indica si el plan actualizó el estado en tiempo real
generated_at	TIMESTAMPTZ	NOT NULL, DEFAULT now()	Momento en que fue generado el plan

7.3 Diagrama entidad relacion

El siguiente diagrama representa visualmente todas las entidades del sistema, sus atributos y las relaciones que existen entre ellas. Las tablas están agrupadas por color según su grupo funcional: Routing (azul), Video/Cámaras (naranja), Detección IA (verde) y Estado/Despacho (morado).

Ilustración 35. Diagrama entidad relación. Fuente propia



Cada caja representa una tabla. Las filas internas muestran sus columnas con el tipo de dato correspondiente. Las etiquetas PK (Primary Key) marcan las claves primarias en rojo y FK (Foreign Key) marcan las claves foráneas en morado. Las líneas entre tablas indican relaciones, anotadas con su cardinalidad (1:1, 1:N, N:M) y el nombre semántico de la relación. La línea punteada entre `station_states` y `dispatch_plans` indica una relación opcional.

7.4 Script SQL de Creación

A continuación se presenta el script completo de creación de la base de datos para PostgreSQL 15+. El script incluye la definición de todas las tablas con sus restricciones, los índices recomendados y los datos semilla iniciales basados en la configuración actual del sistema.

7.4.1 TABLA: `train_models`

Tabla 36. TABLA: `train_models`. Fuente propia.

```
CREATE TABLE train_models (
  id          VARCHAR(50)  PRIMARY KEY,
  name        VARCHAR(100) NOT NULL,
  wagon_count SMALLINT     NOT NULL CHECK (wagon_count
>= 1),
  capacity_per_wagon SMALLINT NOT NULL CHECK
(capacity_per_wagon >= 1),
  total_capacity SMALLINT   GENERATED ALWAYS AS
(wagon_count * capacity_per_wagon) STORED,
  created_at   TIMESTAMPTZ NOT NULL DEFAULT now()
);

COMMENT ON TABLE train_models IS 'Catálogo de modelos de tren
disponibles para despacho';
COMMENT ON COLUMN train_models.id IS 'Identificador único:'
```

```
light, medium, heavy';
```

7.4.2 TABLA: cameras

Tabla 37. TABLA: train_models. Fuente propia

```
CREATE TABLE cameras (  
  id          VARCHAR(50)  PRIMARY KEY,  
  name        VARCHAR(100) NOT NULL,  
  source_path VARCHAR(255) NOT NULL,  
  resolution_width  SMALLINT  DEFAULT 1920,  
  resolution_height SMALLINT  DEFAULT 1080,  
  fps          SMALLINT  DEFAULT 30,  
  is_active     BOOLEAN    NOT NULL DEFAULT TRUE,  
  created_at    TIMESTAMPTZ NOT NULL DEFAULT now()  
);  
  
COMMENT ON TABLE  cameras IS 'aca van los dispositivos de  
camaras';  
COMMENT ON COLUMN cameras.source_path IS
```

7.4.3 TABLA: stations

Tabla 38. TABLA: stations. Fuente propia

```
CREATE TABLE stations (  
  id          VARCHAR(50)  PRIMARY KEY,  
  name        VARCHAR(100) NOT NULL,  
  line        VARCHAR(50)  NOT NULL,  
  camera_id   VARCHAR(50)  NOT NULL UNIQUE,  
  travel_time_seconds  INTEGER  NOT NULL CHECK  
(travel_time_seconds >= 0),  
  created_at  TIMESTAMPTZ  NOT NULL DEFAULT now(),  
  updated_at  TIMESTAMPTZ  NOT NULL DEFAULT now(),  
  
  CONSTRAINT fk_stations_camera  
    FOREIGN KEY (camera_id) REFERENCES cameras(id)  
    ON DELETE RESTRICT
```

```

        ON UPDATE CASCADE
    );

COMMENT ON TABLE stations IS 'Configuración de estaciones con
su línea y cámara vinculada';
COMMENT ON COLUMN stations.travel_time_seconds IS 'Tiempo
promedio en segundos hacia la siguiente estación';

```

7.4.4 TABLA: station_train_support

Tabla 39. TABLA: station_train_support. Fuente propia

```

CREATE TABLE station_train_support (
    station_id          VARCHAR(50)  NOT NULL,
    train_model_id      VARCHAR(50)  NOT NULL,
    assigned_at         TIMESTAMPTZ  NOT NULL DEFAULT now(),

    PRIMARY KEY (station_id, train_model_id),

    CONSTRAINT fk_sts_station
        FOREIGN KEY (station_id) REFERENCES stations(id)
        ON DELETE CASCADE
        ON UPDATE CASCADE,

    CONSTRAINT fk_sts_train
        FOREIGN KEY (train_model_id) REFERENCES train_models(id)
        ON DELETE RESTRICT
        ON UPDATE CASCADE
);

COMMENT ON TABLE station_train_support IS 'Relación N:M entre
estaciones y modelos de tren soportados';

```

7.4.5 TABLA: detection_events

Tabla 40. TABLA: detection_events. Fuente propia

```

CREATE TABLE detection_events (
    id                  BIGSERIAL     PRIMARY KEY,

```

```

    camera_id      VARCHAR(50)  NOT NULL,
    person_count   SMALLINT     NOT NULL CHECK (person_count >=
0),
    total_objects  SMALLINT     NOT NULL CHECK (total_objects
>= 0),
    avg_confidence NUMERIC(5,4) CHECK (avg_confidence >= 0.0
AND avg_confidence <= 1.0),
    frame_timestamp TIMESTAMPTZ NOT NULL,
    raw_detections JSONB,
    created_at     TIMESTAMPTZ  NOT NULL DEFAULT now(),

    CONSTRAINT fk_detection_camera
        FOREIGN KEY (camera_id) REFERENCES cameras(id)
        ON DELETE RESTRICT
        ON UPDATE CASCADE
);

COMMENT ON TABLE detection_events IS 'Histórico de detecciones
de YOLO por frame procesado';
COMMENT ON COLUMN detection_events.raw_detections IS 'JSON con
bounding boxes, clases y confianza por objeto detectado';

```

7.4.6 TABLA: dispatch_plans

Tabla 41. TABLA: dispatch_plans. Fuente propia

```

CREATE TABLE dispatch_plans (
    id                      BIGSERIAL      PRIMARY KEY,
    station_id             VARCHAR(50)    NOT NULL,
    camera_id              VARCHAR(50)    NOT NULL,
    train_model_id         VARCHAR(50)    NOT NULL,
    passenger_count        INTEGER        NOT NULL CHECK
(passenger_count >= 0),
    trains_required        SMALLINT       NOT NULL CHECK
(trains_required >= 0),
    total_capacity         INTEGER        NOT NULL CHECK
(total_capacity >= 0),
    surplus_capacity       INTEGER        NOT NULL CHECK
(surplus_capacity >= 0),
    estimated_cycle_time_seconds INTEGER    NOT NULL CHECK
(estimated_cycle_time_seconds >= 0),

```

```

        status                VARCHAR(100) NOT NULL,
        persist                BOOLEAN     NOT NULL DEFAULT
TRUE,
        generated_at          TIMESTAMPTZ  NOT NULL DEFAULT
now(),

        CONSTRAINT fk_dispatch_station
            FOREIGN KEY (station_id) REFERENCES stations(id)
            ON DELETE RESTRICT
            ON UPDATE CASCADE,

        CONSTRAINT fk_dispatch_camera
            FOREIGN KEY (camera_id) REFERENCES cameras(id)
            ON DELETE RESTRICT
            ON UPDATE CASCADE,

        CONSTRAINT fk_dispatch_train
            FOREIGN KEY (train_model_id) REFERENCES train_models(id)
            ON DELETE RESTRICT
            ON UPDATE CASCADE
    );

COMMENT ON TABLE dispatch_plans IS 'Histórico de planes de
despacho generados por el motor de optimización';

```

7.4.7 TABLA: station_states

Tabla 42. TABLA: station_states. Fuente propia

```

CREATE TABLE station_states (
    station_id          VARCHAR(50)  PRIMARY KEY,
    last_passenger_count INTEGER      CHECK
(last_passenger_count >= 0),
    last_updated         TIMESTAMPTZ,
    last_dispatch_id     BIGINT,

    CONSTRAINT fk_state_station
        FOREIGN KEY (station_id) REFERENCES stations(id)
        ON DELETE CASCADE
        ON UPDATE CASCADE,

```

```

        CONSTRAINT fk_state_dispatch
            FOREIGN KEY (last_dispatch_id) REFERENCES
dispatch_plans(id)
            ON DELETE SET NULL
            ON UPDATE CASCADE
    );

COMMENT ON TABLE station_states IS 'Estado operativo actual por
estación (1 fila por estación)';

```

7.4.8 Indices de rendimiento.

Tabla 43. TABLA: Indices de rendimiento. Fuente propia

```

CREATE INDEX idx_detection_camera_time
    ON detection_events (camera_id, frame_timestamp DESC);
CREATE INDEX idx_detection_timestamp
    ON detection_events (frame_timestamp DESC);
CREATE INDEX idx_detection_raw_gin
    ON detection_events USING GIN (raw_detections);
CREATE INDEX idx_dispatch_station_time
    ON dispatch_plans (station_id, generated_at DESC);
CREATE INDEX idx_dispatch_generated
    ON dispatch_plans (generated_at DESC);

```

7.5 Datos semilla - Datos con base en la configuración actual del backend.

7.5.1 Modelos de tren

Tabla 44. TABLA: Modelos de tren. Fuente propia.

```

INSERT INTO train_models (id, name, wagon_count,
capacity_per_wagon) VALUES
    ('light', 'Tren Ligero (1 vagón)', 1, 5),
    ('medium', 'Tren Medio (2 vagones)', 2, 5),
    ('heavy', 'Tren Pesado (3 vagones)', 3, 5);

```

7.5.2 Cámaras

Tabla 45. TABLA:Cámaras. Fuente propia.

```
INSERT INTO cameras (id, name, source_path) VALUES
('usb0', 'Cámara USB 0', '/dev/video0'),
('usb1', 'Cámara USB 1', '/dev/video1'),
('video1', 'Stream Central',
'src/app/assets/video1.mp4'),
('video2', 'Stream Norte',
'src/app/assets/video2.mp4'),
('video3', 'Stream Este',
'src/app/assets/video3.mp4'),
('video4', 'Stream Oeste',
'src/app/assets/video4.mp4');
```

7.5.3 Estaciones

Tabla 46. TABLA:Estaciones. Fuente propia.

```
INSERT INTO stations (id, name, line, camera_id,
travel_time_seconds) VALUES
('station_sur_usb0', 'Estación Sur USB0', 'Línea 3', 'usb0',
210),
('station_sur_usb1', 'Estación Sur USB1', 'Línea 3', 'usb1',
210),
('station_central', 'Estación Central', 'Línea 1',
'video1', 180),
('station_norte', 'Estación Norte', 'Línea 2',
'video2', 240),
('station_east', 'Estación Este', 'Línea 4',
'video3', 270),
('station_west', 'Estación Oeste', 'Línea 5',
'video4', 300);
```

7.5.4 Trenes soportados por cada estación

Tabla 47. TABLA:Trenes soportados por cada estación. Fuente propia.

```
INSERT INTO station_train_support (station_id, train_model_id)
VALUES
  ('station_sur_usb0', 'light'),
  ('station_sur_usb0', 'medium'),
  ('station_sur_usb0', 'heavy'),
  ('station_sur_usb1', 'light'),
  ('station_sur_usb1', 'medium'),
  ('station_sur_usb1', 'heavy'),
  ('station_central', 'light'),
  ('station_central', 'medium'),
  ('station_central', 'heavy'),
  ('station_norte', 'light'),
  ('station_norte', 'medium'),
  ('station_norte', 'heavy'),
  ('station_east', 'light'),
  ('station_east', 'medium'),
  ('station_west', 'light'),
  ('station_west', 'medium');
```

7.5.5 Inicializar estados vacíos para cada estación

Tabla 48. TABLA:Inicializar estados vacíos para cada estación

```
INSERT INTO station_states (station_id) VALUES
  ('station_sur_usb0'),
  ('station_sur_usb1'),
  ('station_central'),
  ('station_norte'),
  ('station_east'),
  ('station_west');
```

7.6 Relaciones entre tablas

El siguiente resumen describe todas las relaciones de clave foránea del esquema

Tabla 49. Resumen de relaciones entre tablas. Fuente propia

Tabla origen	Columna	Tabla destino	Tipo	Comportamiento al eliminar
stations	camera_id	cameras	1:1	RESTRICT
station_train_support	station_id	stations	N:M	CASCADE
station_train_support	train_model_id	train_models	N:M	RESTRICT
detection_events	camera_id	cameras	N:1	RESTRICT
station_states	station_id	stations	1:1	CASCADE
station_states	last_dispatch_id	dispatch_plans	1:1 opcional	SET NULL
dispatch_plans	station_id	stations	N:1	RESTRICT
dispatch_plans	camera_id	cameras	N:1	RESTRICT
dispatch_plans	train_model_id	train_models	N:1	RESTRICT

Capítulo 8. Desarrollo - Backend/API

En la capa backend se integran FastAPI como framework principal, Uvicorn como servidor ASGI, SQLAlchemy para la definición y gestión de modelos de base de datos y Pydantic para la validación de datos.

Capítulo 9. Desarrollo - Frontend / Interfaz visual

Para la interfaz de usuario se utilizan React, Vite y Axios como núcleo funcional. La gestión de versiones se soporta en Git y GitHub, con el uso de submódulos para mantener repositorios independientes pero conectados, lo que permite una estructura modular adecuada para equipos distribuidos.

Capítulo 10. Fase de pruebas

Capítulo 11. Referencias

Li C, Tang J, Zhang J, Zhao Q, Wang L, Li J (2023) Integrated optimization of urban rail transit line planning, timetabling and rolling stock scheduling. PLoS ONE 18(5): e0285932.

<https://doi.org/10.1371/journal.pone.0285932>

Liang, J., Lyu, G., Teo, C., & Gao, Z. (2023). Online Passenger Flow Control in Metro Lines. Operations Research, 71(2), 768-775. <https://doi.org/10.1287/opre.2022.2417>

Hartmann Tolić, I., Nyarko, E. K., & Ceder, A. (2020). Optimization of Public Transport Services to Minimize Passengers' Waiting Times and Maximize Vehicles' Occupancy Ratios. Electronics, 9(2), 360. <https://doi.org/10.3390/electronics9020360>

Lin, C., Wang, K., Wu, D., & Gong, B. (2020). Passenger Flow Prediction Based on Land Use around Metro Stations: A Case Study. Sustainability, 12(17), 6844. <https://doi.org/10.3390/su12176844>

Velastin, S. A., Fernández, R., Espinosa, J. E., & Bay, A. (2020). Detecting, Tracking and Counting People Getting On/Off a Metropolitan Train Using a Standard Video Camera. Sensors, 20(21), 6251. <https://doi.org/10.3390/s20216251>

Long, S., Luan, X., & Corman, F. (2021). Passenger-oriented rescheduling of trains and rolling stock for handling large passenger demand: linearized models with train capacity constraint. Transportmetrica B Transport Dynamics, 9(1), 641-672. <https://doi.org/10.1080/21680566.2021.1932638>

Schettini, T., Jabali, O., & Malucelli, F. (2022). Demand-Driven Timetabling for a Metro Corridor Using a Short-Turning Acceleration Strategy. Transportation Science, 56(4), 919-937. <https://doi.org/10.1287/trsc.2021.1118>

Zhang, H., & Ni, S. (2022). Train Scheduling Optimization for an Urban Rail Transit Line: A Simulated-Annealing Algorithm Using a Large Neighborhood Search Metaheuristic. Journal Of Advanced Transportation, 2022, 1-17. <https://doi.org/10.1155/2022/9604362>

Zhang, R., Yin, S., Ye, M., Yang, Z., & He, S. (2021). A Timetable Optimization Model for Urban Rail Transit with Express/Local Mode. Journal Of Advanced Transportation, 2021, 1-15. <https://doi.org/10.1155/2021/5589185>

TORNS Team. (2026). Presupuesto de Proyecto – TORNS (Gestión de Proyectos) . https://unilibrebog-my.sharepoint.com/:x:/g/personal/johels-ariasb_unilibre_edu_co/IQA3_ZmES9TvTqNbtx8LuEFBAXC_4kkvK_J28BoO7Ld2MGs?e=b2cuQF

TORNS Team. (2026). Estructura de desglose del trabajo - TORNS | Lucidspark. (s. f.). https://lucid.app/lucidspark/a6bcf639-b73e-4a2a-8345-af9c90970d87/edit?viewport_loc=-272%2C12%2C8494%2C3843%2C0_0&invitationId=inv_72d99aed-5ab6-4ab9-86aa-7709c65d0c29